

AI 投资学课程案例报告

基于 Smart Predict--then--Optimize 的多资产 ETF 组合优化策略研究

English Title: A Multi-Asset ETF Portfolio Optimization Case Study
Based on Smart Predict--then--Optimize

课程：投资学

报告人：何剑宝 翁凯乐 冯炳睿 万席宁

完成日期：2026 年 6 月

声明：本报告仅用于课程案例研究，不构成任何投资建议；历史回测结果不代表未来收益。

目录

摘要

Abstract

一、研究引言

二、理论基础与文献综述

三、数据来源、变量构造与数据质量控制

四、模型设定与组合优化方法

五、回测框架与评价指标体系

六、实证结果与投资组合分析

七、稳健性检验、局限性与风险提示

八、研究结论与后续展望

参考文献

附录 A 复现环境、运行命令与输出目录

附录 B 核心代码与实现说明

附录 C 附图汇编

附录 D 附表索引与主要输出表

摘要

本文围绕 AI 技术在投资组合优化中的应用，构建了一条从数据获取、特征工程、预测建模、决策导向组合优化、风险档位组合生成到独立回测审计的完整研究链路。研究对象为 SPY、QQQ、TLT、GLD、VNQ、DBC 六只美国市场 ETF，分别代表美国大盘权益、科技成长权益、长期美国国债、黄金、房地产 REITs 与大宗商品。数据来自 Yahoo Finance 历史行情，并通过 yfinance 与 Yahoo Chart API 进行程序化获取。复跑结果显示，样本期为 2018-01-02 至 2026-05-29，每只 ETF 含 2113 条日度复权收盘价记录，复权价格无缺失、无重复日期、无非正价格，能够支撑课程级别的多资产配置实证研究。

方法上，本文以 Markowitz 均值-方差理论和 Sharpe 风险调整收益指标为投资学基础^{[1][2]}，以 Smart Predict--then--Optimize, SPO 作为机器学习与优化耦合的核心范式^[3]。SPO 的核心思想是：预测模型不应只最小化点预测误差，而应服务于下游优化决策质量。本文参考 Wang 与 Hasuike 关于真实市场组合优化的 SPO 研究^[4]，并基于 PyEPO 的 SPO+ 损失实现本地复现^[5]。在本项目中，传统 PtO 基准采用 Ridge 收益预测加 Markowitz 最大 Sharpe 优化；SPO+ 模型采用 PyTorch 线性预测器与 PyEPO SPOPlus 损失；最终比较基础 SPO+、加入交易费项的 SPO+、加入换手与 L2 正则的 SPO+、PtO-Markowitz、6ETF 等权、SPY 买入持有以及三档 AI 风险组合。

实证部分采用月度调仓和未来 21 个交易日收益作为持有期收益。训练、验证、测试按时间切分，测试期为 2024-01-31 至 2026-04-29，共 28 个调仓月。独立回测层按 0.5% 单边交易成本重新计算净收益、回撤、成本敏感性与未来函数审计。结果显示，SPO+ 换手/L2 约束在非基准策略中综合评分最高，扣成本后总收益为 106.36%，年化收益为 36.41%，Sharpe 为 2.00，最大回撤为 -7.38%；净 Sharpe 最高的策略是 6ETF 等权，Sharpe 为 2.51。这一结果说明，决策导向学习与组合约束结合后能够改善样本期组合表现，但等权多资产配置仍是非常强的风险调整基准，任何 AI 策略都必须在交易成本和回撤约束下接受严格比较。

本文的主要贡献在于：第一，构建了一个可复现、可检查的数据集和回测流程；第二，系统比较 PtO 与 SPO 在组合优化中的差异；第三，将 AI 风险偏好表达约束为结构化 JSON 和程序化校验，而不是停留在主观文本建议；第四，通过图表和附录代码公开核心实现，保证结论可追溯。研究局限包括样本期有限、资产池较小、交易成本简化、未纳入税费与冲击成本、

SPO+ 点预测指标较弱，以及当前 AI 层尚未接入外部大模型。本报告所有结论仅适用于课程案例研究。（源码：<https://github.com/hhhx-lab/-ETF-.git>）

关键词：AI 投资学；ETF；Smart Predict--then--Optimize；SPO+；投资组合优化；回测；风险控制

Abstract

This report studies the application of artificial intelligence to portfolio optimization through a reproducible multi-asset ETF case. The investment universe consists of six U.S. ETFs, namely SPY, QQQ, TLT, GLD, VNQ and DBC, representing broad U.S. equity, technology-oriented equity, long-duration Treasury bonds, gold, real estate and commodities. The dataset is downloaded from Yahoo Finance through yfinance and a Yahoo Chart API fallback. The final dataset covers 2018-01-02 to 2026-05-29 with 2113 adjusted close observations for each ETF and passes the implemented quality checks.

The methodology combines classical portfolio theory and decision-focused machine learning. Markowitz's mean-variance framework and the Sharpe ratio provide the investment foundation, while Smart Predict--then--Optimize (SPO) provides the main learning paradigm. Instead of optimizing point forecast accuracy only, SPO aligns the prediction model with downstream portfolio decision quality. This project implements a local SPO+ reproduction based on PyEPO and compares it with Ridge predict-then-optimize, equal-weight and buy-and-hold benchmarks. In addition, an AI-style risk-profile layer generates low-, medium- and high-risk portfolios under explicit volatility and single-asset constraints.

The backtest uses monthly rebalancing and forward 21-trading-day returns. The out-of-sample period contains 28 monthly observations from 2024-01-31 to 2026-04-29. Under a 0.5% one-way transaction cost assumption, the SPO+ strategy with turnover and L2 regularization achieves the highest non-benchmark quality score, with total net return of 106.36%, annualized return of 36.41%, Sharpe ratio of 2.00 and maximum drawdown of -7.38%. The equal-weight ETF benchmark delivers the highest net Sharpe ratio, highlighting that diversification remains a strong benchmark. All findings are for educational case-study purposes only and do not constitute investment advice.

(code : <https://github.com/hhhx-lab/-ETF-.git>)

Key words: AI investment; ETF; Smart Predict--then--Optimize; SPO+; portfolio optimization; backtesting; risk management

一、研究引言

（一）研究背景与问题提出

人工智能在投资领域的应用通常从收益预测开始。机器学习模型可以处理大量历史价格、成交量、技术指标和横截面特征，并据此输出未来收益、排名或风险状态。然而，投资学关心的终点并不是预测本身，而是预测信号能否转化为可执行、可约束、可复盘的组合权重。一个模型即使在均方误差上略有优势，也可能因为预测误差被组合优化器放大而导致过度集中、换手过高或回撤恶化；反过来，一个点预测指标并不突出的模型，也可能在下游组合约束中形成更有用的排序信号。因此，AI 投资案例不能只汇报准确率，还必须回到投资组合决策质量。

ETF 是课程案例研究中较合适的资产载体。与个股相比，ETF 的资产类别更清晰、流动性通常更好、分散化属性更强，也更便于从投资学角度解释收益来源和风险暴露。本文选取六只代表性 ETF：SPY 代表美国大盘股票，QQQ 代表科技成长股，TLT 代表长期美国国债，GLD 代表黄金，VNQ 代表房地产 REITs，DBC 代表大宗商品。这样的资产池虽然不完整，但覆盖了权益、债券、贵金属、房地产和商品等主要风险来源，可以支撑关于多资产配置、风险分散和模型优化的讨论。

传统投资组合优化常采用“先预测、再优化”的 PtO 思路。该流程先估计资产收益或风险，再将估计值输入均值-方差、最大 Sharpe 或其他优化模型。PtO 的优势是结构清楚、易于解释，但其训练目标通常与最终投资目标分离。SPO 框架则试图把优化问题嵌入学习目标，使模型在训练阶段就感知预测误差对最终权重的影响^[3]。本文围绕这一范式，结合本地项目代码与实证输出，重新撰写一篇专业标准的投资学课程案例报告。

（二）研究目标与核心问题

本文主要回答四个问题。第一，如何构建一个能够支持 AI 投资组合研究的高质量 ETF 数据集？这包括数据来源、价格字段选择、复权收盘价处理、收益率计算、

特征构造、标签生成以及未来函数控制。第二，SPO 与传统 PtO 在投资组合场景中
有何理论差异？为什么“预测更准”不必然等于“组合更优”？第三，在本地 6 ETF 数据
集和 28 个月样本外测试期中，不同策略在净收益、波动率、Sharpe、最大回撤、换
手率和超额收益方面表现如何？第四，如何将 AI 风险偏好表达转化为可校验的结构
化组合建议，而不是不可复算的文本判断？

（三）研究思路与技术路线

本文技术路线分为六步。第一步，使用 `src/download_data.py` 从 Yahoo Finance
获取六只 ETF 日度行情，并保存原始价格、复权收盘价和日收益率。第二步，使用
`src/features.py` 构造 22 个历史特征和未来 21 日收益标签，形成日度建模样本和月末
调仓样本。第三步，使用 `src/reproduce_spo_paper.py` 训练 Ridge PtO 和 PyEPO SPO+
线性模型，并生成月度组合权重。第四步，使用 `src/generate_ai_risk_profiles.py` 生成
低、中、高三档风险组合，约束年化波动率和单 ETF 权重上限。第五步，使用
`test/backtest_existing_models.py` 独立复算组合指标和未来函数审计，生成累计收益、
回撤、权重热力图、特征重要性和绩效表。第六步，按华东师范大学毕业论文格式
要求整理为正式课程案例报告。

二、理论基础与文献综述

（一）现代投资组合理论

Markowitz 的组合选择理论奠定了现代投资组合理论的基础^[1]。其核心思想是，
投资者面对的不是单一资产的收益最大化问题，而是在多个资产之间权衡预期收益
与风险。均值-方差框架把风险刻画为收益率方差或协方差结构，使资产之间的相关
性成为组合配置的关键因素。在本项目中，6ETF 等权策略的强表现正体现了相关性
分散的力量：即使没有复杂预测模型，权益、债券、黄金、房地产与商品的组合也
能降低净值路径波动。

Sharpe 比率进一步提供了单位风险收益的评价方式^[2]。本文在课程回测中采用年化收益除以年化波动率的简化 Sharpe 口径，同时报告 Sortino、Calmar、最大回撤、胜率和平均换手。这样做的原因是，单一 Sharpe 可能掩盖尾部损失、回撤持续时间和交易成本压力。对 AI 投资组合而言，如果一个策略靠高换手和高集中度获得收益，就必须同时检验扣成本后的收益和回撤，否则容易高估模型价值。

（二）Predict-then-Optimize 范式及其局限

Predict-then-Optimize 是金融机器学习中常见流程。预测阶段可使用线性回归、Ridge、随机森林、神经网络或时间序列模型估计未来收益；优化阶段再将预测收益与协方差矩阵输入 Markowitz 或最大 Sharpe 模型。本文的 PtO 基准对每只 ETF 单独训练 StandardScaler + Ridge(alpha=1.0)，预测未来 21 个交易日收益，再使用历史协方差矩阵求解长期只多最大 Sharpe 权重。

PtO 的局限在于训练目标与决策目标的错位。假设模型 A 的 RMSE 比模型 B 低，但模型 A 在排名第一和第二的资产之间经常犯错，而优化器又把大部分权重放在预测排名第一的资产上，那么微小预测误差就可能造成显著组合损失。金融市场低信噪比、非平稳和强噪声的特征会进一步放大这个问题。由此可见，AI 投资的核心不是“模型看起来更聪明”，而是“模型诱导的组合在可交易约束下更稳健”。

（三）SPO 与决策导向学习

Elmachtoub 与 Grigas 提出的 SPO 框架正是为了解决预测与优化分离问题^[3]。SPO 不再只用预测误差训练模型，而是让预测器直接考虑下游优化问题。直观地说，SPO 希望预测误差在不影响决策的方向上可以被容忍，而会导致错误决策的预测误差应被更强惩罚。SPO+ 是 SPO 损失的可训练 surrogate，能在 PyTorch 等框架中通过优化层反馈梯度。

Wang 与 Hasuike 的 2026 年论文把 SPO 范式应用于真实市场 ETF 组合优化^[4]。该论文指出，收益预测准确率提升并不总能带来组合决策质量提升，尤其是在存在交易成本、换手约束和市场非平稳性的情况下。论文采用线性预测器以保持解释性，并引入交易费、换手控制、L2 正则、RobustSPO 和 SoftmaxDFL 等扩展。本文吸收其核心思想，但根据本地课程项目约束做了简化：本项目复现 SPO+ 主路径，使用 PyEPO 实现 SPOPlus 损失，暂未完整复现 RobustSPO、SoftmaxDFL 和 Optuna 滚动调参。

（四）PyEPO 框架与可复现实现

PyEPO 是一个基于 PyTorch 的端到端 predict-then-optimize 工具库，支持多种线性和整数规划问题的决策导向学习^[5]。本项目已下载 PyEPO 官方代码到 external/PyEPO/，并在 etf-spo conda 环境中以 editable 方式安装。由于原论文 arXiv 源码包只包含论文 LaTeX 与图表，没有提供可运行实验代码，本项目使用 PyEPO 官方实现作为 SPO+ 损失基础。

本项目的自定义优化模型为 MaxReturnOptModel。在长期只多、权重和为 1 的 MaxReturn 问题中，如果不加入额外约束，最优解会将全部权重给预测收益最高的资产。这种解法简洁，但容易造成过度集中。因此项目在后续权重生成中加入交易费、换手惩罚和 L2 正则，形成 spo_plus_turnover_l2 策略。它不是简单追逐最高预测资产，而是在收益、换手和权重分散之间做数值权衡。

（五）ETF 资产池构成与代表性

本文数据来自 Yahoo Finance 和 yfinance^{[6][7]}，ETF 资产类别则由发行方官方说明确认。SPY 由 State Street 提供，跟踪 S&P 500 指数，代表美国大盘权益^[8]；QQQ 由 Invesco 提供，基于 Nasdaq-100 指数，代表科技成长和非金融大型成长股暴露^[9]；TLT 由 iShares 提供，跟踪 20 年以上美国国债指数，代表久期较长的利率风险

暴露^[10]；GLD 由 World Gold Council 相关平台提供，目标是反映黄金价格表现，代表贵金属和避险资产^[11]；VNQ 由 Vanguard 提供，跟踪美国房地产相关指数，代表房地产权益风险^[12]；DBC 由 Invesco 提供，跟踪多元商品指数，代表商品周期和通胀相关暴露^[13]。

三、数据来源、变量构造与数据质量控制

（一）数据来源与采集机制

本项目使用 Yahoo Finance historical market data 作为历史行情来源，并通过 `yfinance.download()` 进行程序化获取。如果出现限流或网络异常，脚本自动切换到 Yahoo Chart API 备用通道。最近复跑时，`yfinance` 主通道返回限流提示，脚本成功切换至 Chart API，并通过数据质量检查。所有下载数据保存在 `data/raw/etf_prices_raw.csv`，复权收盘价保存在 `data/processed/prices_adj_close.csv`，日收益率保存在 `data/processed/daily_returns.csv`。

表 1 数据源与资产池说明 Table 1 Data source and ETF universe

代码	官方名称	资产类别	组合含义
SPY	State Street SPDR S&P 500 ETF Trust	美国大盘股票	S&P 500 大盘权益代表
QQQ	Invesco QQQ Trust	科技成长股票	Nasdaq-100 成长股暴露
TLT	iShares 20+ Year Treasury Bond ETF	长期美国国债	久期与利率风险暴露
GLD	SPDR Gold Shares	黄金	贵金属与避险资产
VNQ	Vanguard Real Estate ETF	房地产 REITs	房地产权益风险
DBC	Invesco DB Commodity Index Tracking Fund	大宗商品	商品周期和通胀相关暴露

（二）数据字段、复权价格与收益率处理

原始数据包含 Open、High、Low、Close、Adj Close、Volume 六类字段。本文使用 Adj Close 计算收益率，因为复权收盘价已经考虑现金分红和拆分影响，更适合

长期持有和多资产回测。日收益率定义为复权收盘价的一阶百分比变化。收益率首日自然为空，这是由 pct_change() 决定的计算结果，不属于价格缺失。

表 2 数据质量检查结果 Table 2 Data quality check

检查项	结果
样本区间	2018-01-02 至 2026-05-29
每只 ETF 价格行数	2113
复权价格缺失值	0
重复日期	0
非正价格	0
原始价格表形状	(2113, 36)
复权收盘价矩阵形状	(2113, 6)
质量总判定	通过

（三）特征工程与标签定义

特征工程脚本 src/features.py 生成 22 个历史特征，覆盖短中长期动量、波动率、下行波动率、滚动回撤、均线比率、成交量变化、风险调整收益、横截面排名和横截面 z-score。标签包括未来 21 个交易日收益、未来收益排名、是否跑赢横截面中位数、是否进入未来前二名、未来收益是否为正。

选择未来 21 个交易日作为标签，是为了近似一个月持有期，与月度调仓频率一致。月末调仓样本由日度建模样本筛选每月最后一个可交易日得到。当前复跑生成 11832 行日度建模样本、390 行月末调仓样本。训练集、验证集和测试集按时间划分：训练集不晚于 2022-12-31，验证集为 2023 年，测试集从 2024 年开始。

表 3 建模数据集摘要 Table 3 Modeling dataset summary

项目	数值
特征数量	22
标签数量	5
首个可用特征日期	2018-06-25
最后可用标签日期	2026-04-29
日度样本行数	11832
月末样本行数	390

训练月末样本	228
验证月末样本	54
测试月末样本	108
建模特征缺失	无
建模标签缺失	无

（四）时间切分与未来函数控制

未来函数是金融回测中最常见也最严重的问题之一。本文从三个层面控制未来函数。第一，特征计算只使用样本日及以前的价格、收益和成交量；第二，StandardScaler 只在训练集上拟合，验证集和测试集仅使用训练集参数转换；第三，回测收益使用已经存储的未来 21 日标签，且回测日期与测试标签日期严格一致。独立审计文件 test/backtest_outputs/lookahead_audit.json 显示所有检查均通过。

四、模型设定与组合优化方法

（一）PtO-Markowitz 基准模型

PtO-Markowitz 策略分为预测和优化两步。预测阶段，对每只 ETF 分别训练 Ridge 回归模型，输入为 22 个历史特征，输出为未来 21 日收益。优化阶段，将预测收益年化，并结合最近 60 个交易日的历史协方差矩阵，求解长期只多最大 Sharpe 组合。约束条件为所有权重非负且权重和为 1。

PtO 基准的意义在于提供一个传统机器学习投资流程对照。它能够体现预测模型和 Markowitz 优化的结合，但也暴露了高换手和回撤问题。当前独立回测显示，PtO-Markowitz 扣成本后总收益 22.00%，最大回撤 -15.90%，平均换手 126.39%，说明成本敏感性较高。

（二）SPO+ 决策导向学习模型

SPO+ 模型使用 PyTorch 线性层从月末特征向量预测六只 ETF 的未来收益分数，并用 PyEPO 的 SPOPlus 损失训练。训练时，模型不只是拟合真实收益，而是通

过优化模型感知预测值导致的组合决策差异。主脚本默认训练 240 轮，batch size 为 16，学习率为 0.01，优化器为 Adam，weight decay 为 1e-4。

基础 SPO+ 的 MaxReturn 权重会高度集中；这符合线性目标在 simplex 约束下的数学性质，但不一定符合投资实践。为此，项目设置了三类 SPO 策略：基础 spo_plus、加入交易费项的 spo_plus_fee、加入交易费、换手惩罚和 L2 权重正则的 spo_plus_turnover_l2。其中最后一个策略是本文重点讨论的 SPO 类组合，因为它显式处理交易摩擦和权重稳定性。

（三）风险偏好约束下的 AI 组合生成

AI 风险组合层不是直接调用大模型生成主观建议，而是构造了一套可被大模型接入、也能由程序化优化器复现的结构化流程。输入包括 SPO 候选组合、ETF 单体风险收益指标、SPO 最新排名和三档风险约束。输出必须为 JSON，包括权重、预期收益、预期波动、历史风险检查、校验状态、风险说明和配置理由。

三档风险约束为：低风险年化波动率不超过 10%、单只 ETF 权重不超过 35%；中风险年化波动率不超过 15%、单只 ETF 权重不超过 45%；高风险年化波动率不超过 22%、单只 ETF 权重不超过 60%。若某月无法完全满足波动率约束，系统回退到同一单资产上限下的最低波动组合，并记录 fallback_to_minimum_volatility。当前复跑中权重和、非负和单资产上限全部通过，风险约束有 2 次回退且已标注。

表 4 最新 AI 三档风险组合 Table 4 Latest AI risk-profile portfolios

风险档位	波动上限	单 ETF 上限	最新权重	预期年化收益	预期年化波动	预期 Sharpe
低风险	10.00%	35.00%	QQQ 35.0%, VNQ 30.0%, DBC 35.0%	38.53%	9.24%	4.17
中风险	15.00%	45.00%	QQQ 45.0%, VNQ 10.0%,	46.44%	10.56%	4.40

			DBC 45.0%			
			QQQ			
高风险	22.00%	60.00%	40.0%,	53.28%	12.08%	4.41
			DBC 60.0%			

（四）交易成本、换手惩罚与正则化约束

本文主口径采用 0.5% 单边交易成本。换手率定义为本期权重与上期权重差的绝对值之和。净收益计算公式为： $\text{net_return_t} = \text{gross_return_t} - \text{turnover_t} * 0.005$ 。这一路径虽简化了真实交易中的买卖价差、市场冲击、税费和基金申赎机制，但足以在课程案例中体现高换手策略的成本压力。

将权重上限、换手控制和交易成本写入组合优化，也与大规模组合中约束可以缓解估计误差影响的经验研究相一致^[14]，并与把交易成本纳入多期组合优化的研究方向相呼应^[15]。

交易成本对结果影响明显。基础 SPO+ 扣成本前总收益 86.88%，扣成本后总收益降至 55.63%；PtO-Markowitz 扣成本前总收益 45.37%，扣成本后仅 22.00%。这说明 AI 投资策略如果忽略换手成本，容易产生过度乐观结论。

五、回测框架与评价指标体系

（一）样本外区间与月度调仓规则

回测采用月度调仓。每个调仓日为当月最后一个可交易日，模型读取截至该日可观察的特征，生成未来 21 个交易日收益预测或决策分数，再据此构建组合权重。组合持有至下一期，用未来 21 日真实收益计算本期实现收益。测试期为 2024-01-31 至 2026-04-29，共 28 个调仓月。

这种回测设计与课程项目的数据规模匹配。日度调仓可能放大噪声和交易成本，季度调仓又可能过度稀疏。月度调仓在学术研究和资产配置实践中都较常见，也与原 SPO 论文中的月度再平衡思想一致^[4]。

（二）绩效评价指标体系

本文使用总收益、年化收益、年化波动率、Sharpe、Sortino、最大回撤、Calmar、胜率、平均换手率和最终财富评价策略。总收益反映累计财富增长，年化收益便于跨周期比较；年化波动率衡量收益不确定性；Sharpe 衡量单位波动收益；Sortino 只考虑下行波动；最大回撤衡量从历史高点到低点的最大损失；Calmar 用年化收益除以最大回撤绝对值；胜率反映正收益月份占比；平均换手体现交易成本压力。

为了避免只对模型策略友好，本文保留两个强基准：6ETF 等权和 SPY 买入持有。等权基准体现多资产分散化，SPY 买入持有体现美股大盘风险暴露。如果 AI/SPO 策略无法在扣成本后超过这些简单基准，其应用价值就需要谨慎评价。

（三）独立复核与未来函数审计

主模型脚本会输出权重和收益，独立回测脚本则读取这些输出重新计算全部指标，并额外输出未来函数审计、随机森林诊断特征重要性、净值曲线、回撤曲线和成本敏感性。审计结果显示：t 月末特征只使用 t 月末及以前数据；标签为未来 21 个交易日收益；组合回测日期与测试标签日期一致；StandardScaler 只在训练集拟合；训练验证期结束早于测试期开始。由此可以认为当前回测没有明显未来函数问题。

六、实证结果与投资组合分析

（一）ETF 资产表现与风险收益特征

如图 1 所示，六只 ETF 从样本起点归一化后的价格或净值走势存在明显差异。权益资产 SPY 与 QQQ 在多数时期表现较强，QQQ 的成长属性带来更高收益和更高波动；TLT 在加息周期承受较大压力；GLD 在避险和通胀环境中具有独立走势；VNQ 对利率和房地产周期敏感；DBC 反映商品周期。资产走势差异说明，跨资产配置比单一资产押注更适合课程研究。

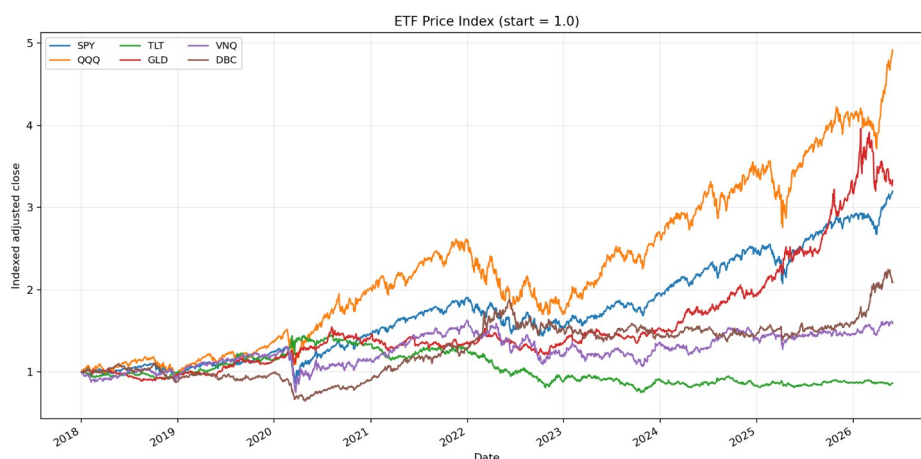


图 1 ETF 价格指数走势 Figure 1 ETF price index

(二) 策略净收益与风险调整表现

由表 5 可见，各策略在扣除 0.5% 单边交易成本后的表现分化明显。SPO+换手/L2 约束 的净收益总额最高，达到 106.36%；其年化收益为 36.41%，Sharpe 为 2.00，最大回撤为 -7.38%。它相对等权基准的年化超额收益为 15.04%，信息比率为 0.82。

表 5 主要策略净收益指标 Table 5 Net backtest metrics

策略	总收益	年化收益	年化波动	Sharpe	最大回撤	胜率	平均换手
SPO+换							
手/L2 约	106.36%	36.41%	18.16%	2.00	-7.38%	75.00%	75.00%
束							
SPO+	55.63%	20.87%	18.14%	1.15	-10.81%	67.86%	132.14%
SPO+交	55.63%	20.87%	18.14%	1.15	-10.81%	67.86%	132.14%
易费							
SPY 买入	55.60%	20.86%	13.74%	1.52	-10.69%	67.86%	3.57%
持有							
6ETF 等	50.49%	19.15%	7.62%	2.51	-4.20%	82.14%	3.57%
权							
AI 低风险	43.62%	16.78%	7.49%	2.24	-5.66%	67.86%	69.01%
组合							
AI 中风险	38.08%	14.83%	9.40%	1.58	-7.52%	75.00%	91.16%
组合							
AI 高风险	30.36%	12.03%	10.41%	1.16	-10.86%	67.86%	103.57%

组合							
PtO-	22.00%	8.90%	10.84%	0.82	-15.90%	64.29%	126.39%
Markowitz							

从图 2 的累计净值曲线可以看到，SPO+换手/L2 约束 在样本期后段形成明显领先；基础 SPO+ 和 SPO+交易费版本走势高度接近，说明当前交易费项没有形成有效区分；等权和 SPY 买入持有的净值更平滑，特别是等权组合在风险调整收益上非常有竞争力。

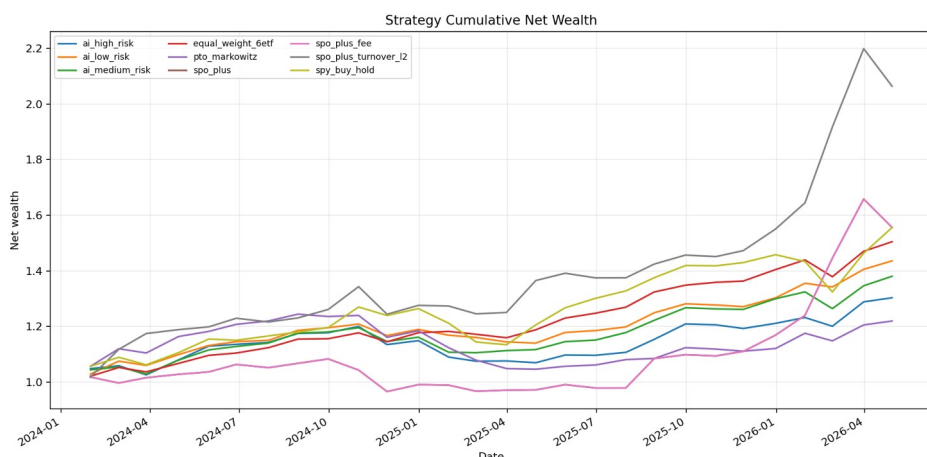


图 2 策略累计收益曲线 Figure 2 Strategy cumulative return curves

（三）回撤控制与下行风险比较

从图 3 的主要策略回撤曲线看，PtO-Markowitz 最大回撤较深，说明传统预测再优化在当前样本下容易受到估计误差和换手影响。高风险 AI 组合也出现较大回撤，其原因是约束更宽、换手更高，并且较多配置到波动资产。相比之下，等权组合回撤最浅之一，这再次说明分散化和低换手在短样本回测中非常重要。

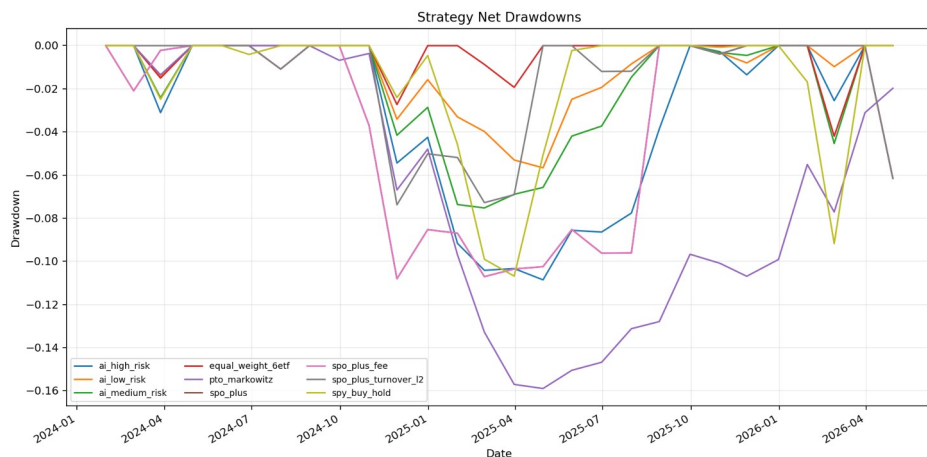


图 3 策略回撤曲线 Figure 3 Strategy drawdown curves

SPO+换手/L2 约束 的最大回撤为 -7.38%，虽然不是全场最低，但在总收益明显领先的同时维持了可接受的回撤水平。AI 低风险组合最大回撤为 -5.66%，体现了风险约束在平滑净值方面的作用。不过 AI 低风险组合扣成本后总收益低于等权基准，说明风险控制本身并不自动创造超额收益。

（四）预测质量与决策质量诊断

由表 6 的预测层指标可见，SPO+ 在 RMSE、MAE 和 R2 上明显弱于 Ridge，测试 Rank IC 接近零。这一现象提醒我们，不能把组合层表现直接解释为模型拥有稳定收益预测能力。更准确的表述是：在当前样本和当前权重生成规则下，SPO+换手/L2 约束组合取得了较好的样本外组合表现；但其预测分数本身仍需更长样本、更严格滚动验证和更多资产检验。

表 6 预测层指标 Table 6 Prediction-level diagnostics

模型	样本	RMSE	MAE	R2	Rank IC
pto_ridge_markowitz	valid	0.0634	0.0534	-0.6168	-0.0952
pyepo_spo_plus_linear	valid	1.4209	1.0903	-811.6444	-0.2048
pto_ridge_markowitz	test	0.0547	0.0407	-0.4967	-0.0041
pyepo_spo_plus_linear	test	1.6039	1.2697	-1285.5018	-0.0041

（五）AI 风险档位组合权重分析

在图 4 的 AI 三档风险组合月度权重热力图中，低风险组合受到 10% 波动上限和 35% 单资产上限约束，配置更分散；中风险组合允许更高单资产权重；高风险组合上限放宽到 60%，因此更容易集中到模型认为预期收益较高的资产。热力图也显示，AI 组合并非静态组合，而是随预测信号和风险约束变化。

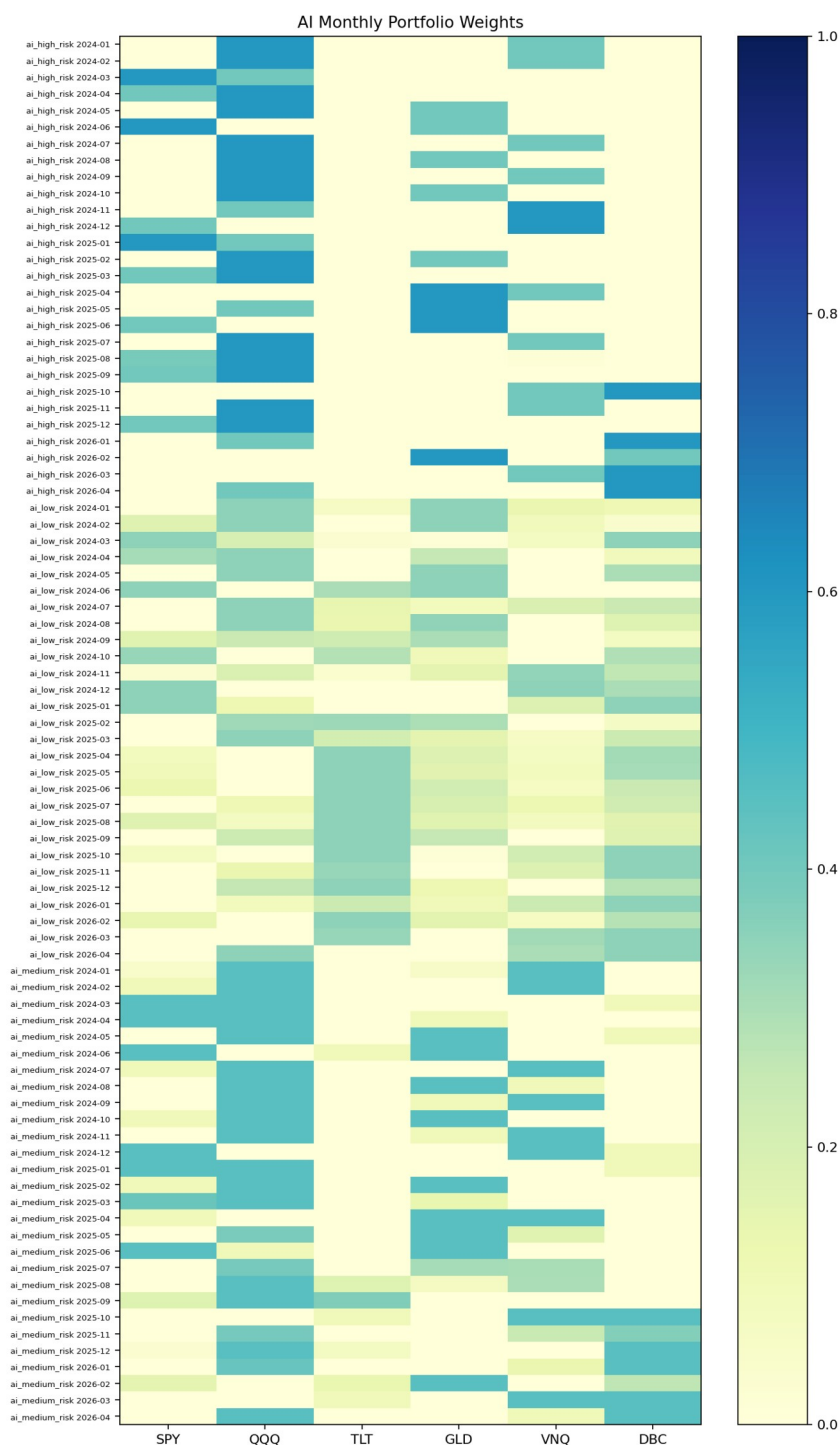


图 4 AI 组合月度权重热力图 Figure 4 AI monthly allocation heatmap

当前最新决策日 2026-04-29 的三档组合均通过权重校验。低风险组合预期年化波动为 9.24%，中风险为 10.56%，高风险为 12.08%。不过，三档组合在扣成本后都没有超过等权基准，这说明可解释的风险偏好组合更适合作为“投资建议展示层”，而不是当前样本下的收益冠军。

（六）诊断性特征重要性分析

随机森林诊断性特征重要性结果见图 5。该模型不参与 SPO/PtO 主回测，只用于帮助解释哪些历史变量对未来 21 日收益标签更有信息。前十特征包括 downside_vol_60d、drawdown_60d、z_vol_60d、drawdown_20d、ma_ratio_60_120、ret_120d 和 volume_chg_20d，说明波动、回撤、趋势和成交量变化都包含一定信息。

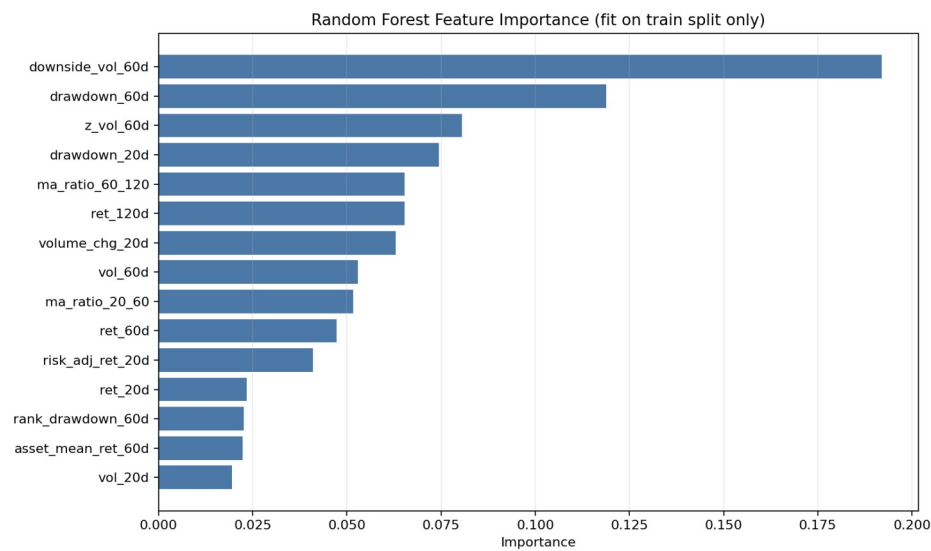


图 5 随机森林特征重要性 Figure 5 Random forest feature importance

表 7 随机森林诊断性特征重要性前十 Table 7 Top 10 diagnostic feature importance

特征	重要性	排名
downside_vol_60d	0.1920	1
drawdown_60d	0.1188	2
z_vol_60d	0.0806	3
drawdown_20d	0.0744	4
ma_ratio_60_120	0.0654	5
ret_120d	0.0653	6
volume_chg_20d	0.0630	7
vol_60d	0.0530	8
ma_ratio_20_60	0.0517	9
ret_60d	0.0473	10

（七）综合绩效可视化比较

此外，图 6 是独立回测脚本生成的绩效表图像，集中展示各策略的收益、波动、Sharpe 和回撤表现。将该图放入报告，是为了让读者在表格之外快速比较模型策略、AI 风险组合和基准组合的整体关系。

Strategy Performance Comparison (Net)

strategy	annual_return	annual_volatility	sharpe	max_drawdown	win_rate	average_turnover	calmar
spo_plus_turnover_l2	36.41%	18.16%	2.00	-7.38%	75.00%	75.00%	4.93
spo_plus	20.87%	18.14%	1.15	-10.81%	67.86%	132.14%	1.93
spo_plus_fee	20.87%	18.14%	1.15	-10.81%	67.86%	132.14%	1.93
spy_buy_hold	20.86%	13.74%	1.52	-10.69%	67.86%	3.57%	1.95
equal_weight_6etf	19.15%	7.62%	2.51	-4.20%	82.14%	3.57%	4.56
ai_low_risk	16.78%	7.49%	2.24	-5.66%	67.86%	69.01%	2.96
ai_medium_risk	14.83%	9.40%	1.58	-7.52%	75.00%	91.16%	1.97
ai_high_risk	12.03%	10.41%	1.16	-10.86%	67.86%	103.57%	1.11
pto_markowitz	8.90%	10.84%	0.82	-15.90%	64.29%	126.39%	0.56

图 6 策略绩效对比表 Figure 6 Strategy performance table

七、稳健性检验、局限性与风险提示

（一）主要研究发现

第一，SPO 类方法在本样本中具有组合层价值。尤其是加入换手和 L2 正则后，策略在净收益和综合评分上领先。这说明，仅将预测分数交给极端 MaxReturn 优化器并不足够，必须加入交易和分散化约束，才能更接近可投资组合。

第二，等权基准非常强。6ETF 等权组合的净 Sharpe 为 2.51，最大回撤仅 -4.20%，平均换手只有 3.57%。这提醒我们，AI 策略不能只和弱基准比较，也不能只展示扣成本前收益。一个专业投资报告必须把简单、低成本、可解释的基准放在同一张表里。

第三，点预测指标与组合绩效并不一致。SPO+ 的点预测误差较大，但带约束的组合表现较好；Ridge 的点预测误差较低，但 PtO-Markowitz 扣成本后表现较弱。这不意味着预测指标无用，而是说明投资场景下必须同时评价预测层和决策层。

第四，AI 风险组合的价值在于结构化表达和可校验输出。本文没有让 AI 直接写“建议买入某 ETF”，而是规定 JSON schema、权重约束和风险复算规则。这样的设计更符合金融场景对审计和可解释性的要求。

（二）稳健性与一致性检验

本文进行了三类稳健性检查。第一，数据质量检查确认价格无缺失、日期一致、无非正价格。第二，未来函数审计确认训练、验证、测试和回测收益之间的时间顺序正确。第三，交易成本敏感性表展示不同成本率下策略表现变化，说明高换手策略对成本假设敏感。虽然这些检查不能证明策略未来有效，但能降低课程报告中常见的数据泄露和口径不一致风险。

（三）研究局限

本文仍有明显局限。第一，测试期只有 28 个调仓月，统计置信度有限。第二，资产池只有六只 ETF，不能代表全球全部资产类别，也不能覆盖行业、风格、期限、信用和地域的完整结构。第三，数据源 Yahoo Finance 和 yfinance 适合课程复现，但正式投资研究应使用交易所、基金公司或专业数据供应商数据复核。第四，交易成本用固定 0.5% 近似，未纳入滑点、市场冲击、税费和流动性冲击。第五，当前实现没有完整复现原论文中的 RobustSPO、SoftmaxDFL 和滚动超参数搜索。第六，AI 层当前是结构化提示词设计和程序化优化器兜底，尚未接入真实大模型 API。

（四）投资风险与适用边界

本报告所有结果均为历史回测，不构成任何投资建议。ETF 价格会受到宏观经济、利率、通胀、政策、地缘政治、市场情绪和流动性等多重因素影响。模型回测收益可能来自样本期偶然结构，未来市场环境变化可能导致策略失效。任何真实投资决策都需要结合投资者风险承受能力、资金期限、税务约束和合规要求。

八、研究结论与后续展望

本文重写并扩展了 AI 投资学课程案例报告，围绕 SPO 决策导向学习构建了完整的多资产 ETF 组合优化研究。数据层面，项目使用 Yahoo Finance 日度历史行情，构建了无缺失、可复现、可追溯的六 ETF 数据集；模型层面，项目比较了 PtO-Markowitz、SPO+、SPO+交易费、SPO+换手/L2 约束、等权、SPY 买入持有和 AI 三档风险组合；回测层面，项目使用月度调仓、未来 21 日持有期收益和 0.5% 单边交易成本，并由独立脚本复算净值、回撤和未来函数审计。

实证结果表明，在当前样本期内，SPO+换手/L2 约束 是综合评分最高的非基准策略，扣成本后总收益 106.36%、年化收益 36.41%、Sharpe 2.00、最大回撤 -7.38%。不过，6ETF 等权组合仍提供最高净 Sharpe，这说明多资产分散化是非常强的投资学基准。本文的核心结论并不是“AI 必然战胜市场”，而是：AI 与 SPO 方法可以把预测信号、优化目标、交易成本和风险偏好放进同一个可复验框架；只有当数据、约束、回测和解释全部经得起检查时，AI 投资案例才具有课程研究意义。

后续研究可以沿四个方向扩展。第一，扩大 ETF 池，加入国际股票、不同期限债券、信用债、行业 ETF、风格 ETF 和波动率产品。第二，引入滚动训练窗口和更长历史样本，检验参数稳定性。第三，完整复现 RobustSPO、SoftmaxDFL 和 Optuna 调参，比较不同决策导向学习结构。第四，接入真实大模型 API，但保留当前程序化校验器，确保 AI 输出始终可复算、可约束、可审计。

参考文献

- [1] Markowitz, H. Portfolio Selection[J]. The Journal of Finance, 1952, 7(1):77-91.
- [2] Sharpe, W. F. The Sharpe Ratio[J]. The Journal of Portfolio Management, 1994/1998, 21(1):49-58.
- [3] Elmachet, A. N., Grigas, P. Smart "Predict, then Optimize"[J]. Management Science, 2022, 68(1):9-26. DOI:10.1287/mnsc.2020.3922.
- [4] Wang, Y., Hasuik, T. Smart Predict--then--Optimize Paradigm for Portfolio Optimization in Real Markets[R]. arXiv:2601.04062, 2026.

- [5] Tang, B., Khalil, E. B. PyEPO: a PyTorch-based End-to-End Predict-then-Optimize Library for Linear and Integer Programming[J]. Mathematical Programming Computation, 2024, 16:297-335. DOI:10.1007/s12532-024-00255-x.
- [6] Yahoo Finance. Historical Prices and Market Data[E]. <https://finance.yahoo.com/>.
- [7] Aroussi, R. yfinance: Download market data from Yahoo! Finance's API[E]. <https://github.com/ranaroussi/yfinance>.
- [8] State Street Investment Management. State Street SPDR S&P 500 ETF Trust (SPY)[E]. <https://www.ssga.com/>.
- [9] Invesco. Invesco QQQ Trust, Series 1 (QQQ)[E]. <https://www.invesco.com/>.
- [10] BlackRock iShares. iShares 20+ Year Treasury Bond ETF (TLT)[E]. <https://www.ishares.com/>.
- [11] World Gold Council. SPDR Gold Shares (GLD)[E]. <https://www.spdrgoldshares.com/>.
- [12] Vanguard. Vanguard Real Estate ETF (VNQ)[E]. <https://investor.vanguard.com/>.
- [13] Invesco. Invesco DB Commodity Index Tracking Fund (DBC)[E]. <https://www.invesco.com/>.
- [14] Jagannathan, R., Ma, T. Risk Reduction in Large Portfolios: Why Imposing the Wrong Constraints Helps[J]. The Journal of Finance, 2003, 58(4):1651-1683.
- [15] Boyd, S. et al. Multi-Period Trading via Convex Optimization[J]. Foundations and Trends in Optimization, 2017, 3(1):1-76.

附录 A 复现环境、运行命令与输出目录

源码: <https://github.com/hhhx-lab/-ETF-.git>

A.1 复现环境

本项目的正式复现环境为 etf-spo Conda 环境。环境文件位于 environment/etf-spo.yml , 核心依赖包括 Python、pandas、numpy、scikit-learn、scipy、PyTorch、PyEPO、matplotlib、seaborn、yfinance、tqdm、pathos 与 clarabel。

A.2 核心运行命令

```
conda env create -f environment/etf-spo.yml
conda run -n etf-spo python src/download_data.py
conda run -n etf-spo python src/features.py
conda run -n etf-spo python src/reproduce_spo_paper.py
conda run -n etf-spo python src/generate_ai_risk_profiles.py
conda run -n etf-spo python test/backtest_existing_models.py --cost-rate 0.005
```

A.3 输出目录说明

附录 D 列出了全部 CSV、JSON、Markdown 与 PNG 输出产物。核心目录包括：

data/raw/ 存放原始两级表头价格数据；data/processed/ 存放复权价格、日收益率、建模样本与月末调仓样本；outputs/tables/ 存放主模型、SPO 复现、AI 风险组合和数据质量表；test/backtest_outputs/ 存放独立回测报告、指标、审计文件和图表；docs/案例报告/ 存放最终 Markdown、DOCX 与 PDF。

附录 B 核心代码与实现说明

附录 B 摘录的是当前工作区中的真实核心源码。为避免报告过度冗长，附录保留数据下载、特征工程、SPO 训练、组合优化、AI 风险组合和独立回测的关键函数；完整代码以附表 D-1 中的路径为准。

附表 B-1 核心代码文件与函数索引 Appendix Table B-1 Core code file and function index

模块	源码路径	核心函数或类
数据下载与质量控制	src/download_data.py	download_with_yfinance, download_with_chart_api, prepare_outputs, quality_checks
特征工程与标签生成	src/features.py	build_wide_features, build_datasets
SPO+训练与组合生成	src/reproduce_spo_paper.py	MaxReturnOptModel, train_spo_plus, long_only_optimize, build_strategy_weights, backtest
AI 风险档位组合	src/generate_ai_risk_profiles.py	optimize_profile, build_ai_weights, validate_ai_outputs
独立回测与审计	test/ backtest_existing_models.py	build_lookahead_audit, evaluate_returns, calculate_metrics, rank_models, save_all_plots

B.1 数据下载与质量控制

数据层脚本先尝试 yfinance 主通道，若网络或限流导致失败，则切换到 Yahoo Chart API 备用通道。只有当质量检查通过后，脚本才会写出原始价格、复权收盘价、收益率和元数据。

```
def download_with_yfinance(
```

```

    tickers: list[str], start: str, end: str, interval: str
) -> pd.DataFrame:
    raw = yf.download(
        tickers,
        start=start,
        end=end,
        interval=interval,
        auto_adjust=False,
        progress=False,
        group_by="column",
        threads=True,
    )
    return normalize_download(raw, tickers)

def download_with_chart_api(
    tickers: list[str], start: str, end: str, interval: str
) -> pd.DataFrame:
    frames = [fetch_chart_ticker(ticker, start, end, interval) for ticker in tickers]
    raw = pd.concat(frames, axis=1).sort_index(axis=1, level=[0, 1])
    raw.index.name = "Date"
    return raw

def prepare_outputs(
    raw: pd.DataFrame, tickers: list[str]
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict[str, object]]:
    raw = normalize_download(raw, tickers)
    adj_close = extract_field(raw, "Adj Close", tickers)
    daily_returns = adj_close.pct_change()
    quality, summary = quality_checks(raw, adj_close, daily_returns, tickers)
    return adj_close, daily_returns, quality, summary

def quality_checks(
    raw: pd.DataFrame,
    adj_close: pd.DataFrame,
    daily_returns: pd.DataFrame,
    tickers: list[str],
) -> tuple[pd.DataFrame, dict[str, object]]:
    duplicate_dates = int(adj_close.index.duplicated().sum())
    shared_row_count = int(adj_close.dropna(how="any").shape[0])

    rows = []
    row_counts = {}
    missing_counts = {}
    first_dates = {}
    last_dates = {}

    for ticker in tickers:
        price = adj_close[ticker]
        returns = daily_returns[ticker]
        valid_price = price.dropna()

        row_count = int(valid_price.shape[0])
        row_counts[ticker] = row_count
        missing_count = int(price.isna().sum())
        missing_counts[ticker] = missing_count
        first_date = valid_price.index.min()
        last_date = valid_price.index.max()
        first_dates[ticker] = first_date.strftime("%Y-%m-%d") if pd.notna(first_date)
    else ""
    last_dates[ticker] = last_date.strftime("%Y-%m-%d") if pd.notna(last_date)
    else ""

    non_positive_prices = int((valid_price <= 0).sum())
    extreme_return_count = int((returns.abs() > 0.20).sum())
    max_abs_return = float(returns.abs().max(skipna=True))

    rows.append(
        {
            "ticker": ticker,
            "first_date": first_dates[ticker],

```

```

        "last_date": last_dates[ticker],
        "price_rows": row_count,
        "missing_adj_close": missing_count,
        "non_positive_adj_close": non_positive_prices,
        "daily_return_rows": int(returns.dropna().shape[0]),
        "missing_daily_returns": int(returns.isna().sum()),
        "extreme_abs_return_gt_20pct": extreme_return_count,
        "max_abs_daily_return": max_abs_return,
        "min_adj_close": float(valid_price.min()) if not valid_price.empty
    else np.nan,
        "max_adj_close": float(valid_price.max()) if not valid_price.empty
    else np.nan,
        "period_total_return": float(valid_price.iloc[-1] /
valid_price.iloc[0] - 1)
        if row_count >= 2
        else np.nan,
        "period_max_drawdown": max_drawdown(returns),
    }
)

quality = pd.DataFrame(rows)
summary = {
    "tickers": tickers,
    "all_row_counts_equal": len(set(row_counts.values())) == 1,
    "row_counts": row_counts,
    "missing_counts": missing_counts,
    "first_dates": first_dates,
    "last_dates": last_dates,
    "expected_first_trading_day": EXPECTED_FIRST_TRADING_DAY,
    "expected_last_trading_day": EXPECTED_LAST_TRADING_DAY,
    "all_start_dates_match_expected": all(
        d == EXPECTED_FIRST_TRADING_DAY for d in first_dates.values()
    ),
    "all_end_dates_match_expected": all(
        d == EXPECTED_LAST_TRADING_DAY for d in last_dates.values()
    ),
    "duplicate_dates": duplicate_dates,
    "shared_complete_price_rows": shared_row_count,
    "raw_shape": list(raw.shape),
    "adj_close_shape": list(adj_close.shape),
    "daily_returns_shape": list(daily_returns.shape),
    "quality_pass": bool(
        len(set(row_counts.values())) == 1
        and all(v == 0 for v in missing_counts.values())
        and all(d == EXPECTED_FIRST_TRADING_DAY for d in first_dates.values())
        and all(d == EXPECTED_LAST_TRADING_DAY for d in last_dates.values())
        and duplicate_dates == 0
        and int(quality["non_positive_adj_close"].sum()) == 0
    ),
}
return quality, summary

```

B.2 特征工程与标签生成

特征工程脚本将宽表价格、收益率和成交量转换为面板式建模数据。所有特征只使用样本日及以前的数据；未来 21 个交易日收益只作为标签和回测收益进入后续流程。

```

def build_wide_features(
    adj: pd.DataFrame,
    returns: pd.DataFrame,
    raw: pd.DataFrame,
    horizon_days: int,
) -> tuple[dict[str, pd.DataFrame], dict[str, pd.DataFrame]]:
    volume = field_from_raw(raw, "Volume")

    ret_5d = adj.pct_change(5)

```

```

ret_20d = adj.pct_change(20)
ret_60d = adj.pct_change(60)
ret_120d = adj.pct_change(120)

vol_20d = returns.rolling(20, min_periods=20).std()
vol_60d = returns.rolling(60, min_periods=60).std()
downside_vol_60d = returns.mask(returns > 0, 0).rolling(60, min_periods=60).std()

drawdown_20d = rolling_max_drawdown(returns.fillna(0), 20)
drawdown_60d = rolling_max_drawdown(returns.fillna(0), 60)

ma_20 = adj.rolling(20, min_periods=20).mean()
ma_60 = adj.rolling(60, min_periods=60).mean()
ma_120 = adj.rolling(120, min_periods=120).mean()
ma_ratio_20_60 = ma_20 / ma_60 - 1
ma_ratio_60_120 = ma_60 / ma_120 - 1

volume_ma_20 = volume.rolling(20, min_periods=20).mean()
volume_ma_60 = volume.rolling(60, min_periods=60).mean()
volume_chg_20d = volume_ma_20 / volume_ma_60 - 1

risk_adj_ret_20d = ret_20d / vol_20d.replace(0, np.nan)
risk_adj_ret_60d = ret_60d / vol_60d.replace(0, np.nan)

asset_mean_ret_60d = returns.rolling(60, min_periods=60).mean()

wide_features = {
    "ret_5d": ret_5d,
    "ret_20d": ret_20d,
    "ret_60d": ret_60d,
    "ret_120d": ret_120d,
    "vol_20d": vol_20d,
    "vol_60d": vol_60d,
    "downside_vol_60d": downside_vol_60d,
    "drawdown_20d": drawdown_20d,
    "drawdown_60d": drawdown_60d,
    "ma_ratio_20_60": ma_ratio_20_60,
    "ma_ratio_60_120": ma_ratio_60_120,
    "volume_chg_20d": volume_chg_20d,
    "risk_adj_ret_20d": risk_adj_ret_20d,
    "risk_adj_ret_60d": risk_adj_ret_60d,
    "asset_mean_ret_60d": asset_mean_ret_60d,
    "rank_ret_20d": rank_pct(ret_20d, ascending=False),
    "rank_ret_60d": rank_pct(ret_60d, ascending=False),
    "rank_vol_60d": rank_pct(vol_60d, ascending=True),
    "rank_drawdown_60d": rank_pct(drawdown_60d, ascending=False),
    "z_ret_20d": zscore_cross_section(ret_20d),
    "z_ret_60d": zscore_cross_section(ret_60d),
    "z_vol_60d": zscore_cross_section(vol_60d),
}

future_return = adj.shift(-horizon_days) / adj - 1
future_rank = future_return.rank(axis=1, ascending=False, method="first")
cross_section_median = future_return.median(axis=1)

wide_labels = {
    "future_return_21d": future_return,
    "future_rank_21d": future_rank,
    "label_outperform_median_21d": (future_return.gt(cross_section_median,
axis=0)).astype(
        "Int64"
    ),
    "label_top2_21d": (future_rank <= 2).astype("Int64"),
    "label_positive_21d": (future_return > 0).astype("Int64"),
}
return wide_features, wide_labels

def build_datasets(horizon_days: int) -> tuple[pd.DataFrame, pd.DataFrame,
pd.DataFrame, dict]:

```

```

root = project_root()
adj, returns, raw = read_prices(root)
wide_features, wide_labels = build_wide_features(adj, returns, raw, horizon_days)

feature_names = list(wide_features)
label_names = list(wide_labels)
features_panel = wide_to_panel(wide_features, feature_names)
labels_panel = wide_to_panel(wide_labels, label_names)

panel = features_panel.merge(labels_panel, on=["date", "ticker"], how="left")
panel = panel.sort_values(["date", "ticker"]).reset_index(drop=True)
panel = add_splits(panel)
panel = add_month_end_flag(panel, adj)

panel["date"] = pd.to_datetime(panel["date"])
first_feature_date = panel.dropna(subset=feature_names)["date"].min()
last_label_date = panel.dropna(subset=["future_return_21d"])["date"].max()

modeling = panel.dropna(subset=feature_names + ["future_return_21d"]).copy()
modeling[label_names[1:]] = modeling[label_names[1:]].astype(int)

monthly = modeling[modeling["is_month_end_sample"]].copy()

features_only = modeling[["date", "ticker", "split", "is_month_end_sample"] +
feature_names]
labels_only = modeling[["date", "ticker"] + label_names]

summary = {
    "horizon_days": horizon_days,
    "tickers": TICKERS,
    "feature_names": feature_names,
    "label_names": label_names,
    "first_feature_available_date": first_feature_date.strftime("%Y-%m-%d"),
    "last_label_available_date": last_label_date.strftime("%Y-%m-%d"),
    "daily_modeling_shape": list(modeling.shape),
    "monthly_rebalance_shape": list(monthly.shape),
    "features_shape": list(features_only.shape),
    "labels_shape": list(labels_only.shape),
    "split_counts_daily": modeling["split"].value_counts().to_dict(),
    "split_counts_monthly": monthly["split"].value_counts().to_dict(),
    "ticker_counts_daily": modeling["ticker"].value_counts().to_dict(),
    "ticker_counts_monthly": monthly["ticker"].value_counts().to_dict(),
    "label_positive_rates": {
        col: float(modeling[col].mean()) for col in label_names if
col.startswith("label_")
    },
    "monthly_label_positive_rates": {
        col: float(monthly[col].mean()) for col in label_names if
col.startswith("label_")
    },
    "no_missing_features_in_modeling":
bool(modeling[feature_names].isna().sum().sum() == 0),
    "no_missing_labels_in_modeling": bool(modeling[label_names].isna().sum().sum()
== 0),
}
return modeling, monthly, features_only, labels_only, summary

```

B.3 SPO+ 训练、权重生成与净收益回测

SPO 复现脚本包含 PyEPO 优化模型、SPOPlus 损失训练、长期只多优化、月度权重生成和净收益回测。spo_plus_turnover_l2 的关键在于同时考虑预期收益、换手惩罚和 L2 权重正则。

```

class MaxReturnOptModel(optModel):
    """PyEPO optModel for long-only MaxReturn over ETF weights.

```

The paper's baseline SPO formulation starts from MaxReturn. For this linear problem, the optimizer can be solved exactly without a commercial solver: allocate all weight to the largest objective coefficient.

```

"""
modelSense = EPO.MAXIMIZE

def __init__(self, num_assets: int):
    self._objective = np.zeros(num_assets, dtype=np.float32)
    self._num_assets = num_assets
    super().__init__()

def _getModel(self) -> tuple[None, list[int]]:
    return None, list(range(self._num_assets))

@property
def num_cost(self) -> int:
    return self._num_assets

def setObj(self, c: np.ndarray | torch.Tensor | list) -> None:
    arr = np.asarray(c, dtype=np.float32)
    if arr.shape[0] != self._num_assets:
        raise ValueError("Objective length does not match number of assets.")
    self._objective = arr

def solve(self) -> tuple[np.ndarray, float]:
    sol = np.zeros(self._num_assets, dtype=np.float32)
    best_idx = int(np.argmax(self._objective))
    sol[best_idx] = 1.0
    obj = float(self._objective[best_idx])
    return sol, obj

def copy(self) -> "MaxReturnOptModel":
    new_model = MaxReturnOptModel(self._num_assets)
    new_model._objective = self._objective.copy()
    return new_model

def addConstr(self, coefs: np.ndarray | torch.Tensor | list, rhs: float) ->
"MaxReturnOptModel":
    raise NotImplementedError("Closed-form MaxReturn model does not support extra
constraints.")

def relax(self) -> "MaxReturnOptModel":
    return self.copy()

def train_spo_plus(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_valid: np.ndarray,
    y_valid: np.ndarray,
    args: argparse.Namespace,
) -> tuple[LinearReturnModel, dict[str, list[float]]]:
    optmodel = MaxReturnOptModel(len(TICKERS))
    true_sol, true_obj = true_solutions(y_train, optmodel)

    model = LinearReturnModel(x_train.shape[1], len(TICKERS))
    optimizer = torch.optim.Adam(model.parameters(), lr=args.learning_rate,
weight_decay=1e-4)
    mse = nn.MSELoss()
    spo_loss = SPOPlus(optmodel, processes=1)

    dataset = TensorDataset(
        torch.tensor(x_train, dtype=torch.float32),
        torch.tensor(y_train, dtype=torch.float32),
        torch.tensor(true_sol, dtype=torch.float32),
        torch.tensor(true_obj, dtype=torch.float32),
    )

```

```

loader = DataLoader(dataset, batch_size=args.batch_size, shuffle=True)
history = {"epoch": [], "train_loss": [], "valid_mse": [], "valid_rank_ic": []}

for epoch in range(1, args.epochs + 1):
    model.train()
    losses = []
    for xb, yb, solb, objb in loader:
        optimizer.zero_grad()
        pred = model(xb)
        loss = spo_loss(pred, yb, solb, objb) + 0.05 * mse(pred, yb)
        loss.backward()
        optimizer.step()
        losses.append(float(loss.detach()))

    if epoch == 1 or epoch % 20 == 0 or epoch == args.epochs:
        model.eval()
        with torch.no_grad():
            valid_pred = model(torch.tensor(x_valid, dtype=torch.float32)).numpy()
        history["epoch"].append(epoch)
        history["train_loss"].append(float(np.mean(losses)))
        history["valid_mse"].append(float(mean_squared_error(y_valid.reshape(-1),
valid_pred.reshape(-1))))
        history["valid_rank_ic"].append(monthly_rank_ic(y_valid, valid_pred))
    return model, history

def long_only_optimize(
    expected: np.ndarray,
    prev_weight: np.ndarray,
    config: StrategyConfig,
) -> np.ndarray:
    n = len(expected)
    if config.transaction_fee_gamma == 0 and config.turnover_penalty_lambda == 0 and
config.l2_weight_regularization == 0:
        weight = np.zeros(n)
        weight[int(np.argmax(expected))] = 1.0
        return weight

    x0 = prev_weight.copy() if prev_weight.sum() > 0 else np.full(n, 1.0 / n)
    bounds = [(0.0, 1.0) for _ in range(n)]
    constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1.0}]

    def objective(w: np.ndarray) -> float:
        gross = float(expected @ w)
        turnover = float(np.abs(w - prev_weight).sum())
        l2 = float(np.sum(w**2))
        return -gross + config.transaction_fee_gamma * turnover +
config.turnover_penalty_lambda * turnover + config.l2_weight_regularization * l2

    result = minimize(
        objective,
        x0,
        method="SLSQP",
        bounds=bounds,
        constraints=constraints,
        options={"maxiter": 500, "ftol": 1e-10},
    )
    if not result.success:
        return np.full(n, 1.0 / n)
    weight = np.clip(result.x, 0, None)
    return weight / weight.sum()

def build_strategy_weights(
    index: pd.DataFrame,
    y_true: np.ndarray,
    pto_pred: np.ndarray,
    spo_pred: np.ndarray,
    returns: pd.DataFrame,
) -> pd.DataFrame:
    rows = []
    prev: dict[str, np.ndarray] = {}
    strategy_predictions = {

```

```

        "pto_markowitz": pto_pred,
        "spo_plus": spo_pred,
        "spo_plus_fee": spo_pred,
        "spo_plus_turnover_l2": spo_pred,
    }
    strategy_configs = {config.name: config for config in STRATEGIES}

    for i, row in index.reset_index(drop=True).iterrows():
        date = pd.Timestamp(row["date"])
        history = trailing_history(returns, date)
        for strategy, pred in strategy_predictions.items():
            previous = prev.get(strategy, np.zeros(len(TICKERS)))
            if strategy == "pto_markowitz":
                weight = markowitz_weights(pred[i], history)
                config_note = "PtO Ridge prediction with Markowitz max-Sharpe
optimization"
            else:
                weight = long_only_optimize(pred[i], previous,
strategy_configs[strategy])
                config_note = strategy_configs[strategy].description
            prev[strategy] = weight
            for ticker, ticker_weight in zip(TICKERS, weight, strict=False):
                rows.append(
                    {
                        "date": date,
                        "strategy": strategy,
                        "ticker": ticker,
                        "weight": float(ticker_weight),
                        "predicted_return_21d": float(pred[i, TICKERS.index(ticker)]),
                        "actual_future_return_21d": float(y_true[i,
TICKERS.index(ticker)]),
                        "strategy_note": config_note,
                    }
                )

    for benchmark in ["equal_weight_6etf", "spy_buy_hold"]:
        weight = np.full(len(TICKERS), 1.0 / len(TICKERS))
        if benchmark == "spy_buy_hold":
            weight = np.zeros(len(TICKERS))
            weight[TICKERS.index("SPY")] = 1.0
        for ticker, ticker_weight in zip(TICKERS, weight, strict=False):
            rows.append(
                {
                    "date": date,
                    "strategy": benchmark,
                    "ticker": ticker,
                    "weight": float(ticker_weight),
                    "predicted_return_21d": np.nan,
                    "actual_future_return_21d": float(y_true[i,
TICKERS.index(ticker)]),
                    "strategy_note": "benchmark",
                }
            )

    return pd.DataFrame(rows)

def backtest(weights: pd.DataFrame, cost_rate: float = DEFAULT_COST_RATE) ->
tuple[pd.DataFrame, pd.DataFrame]:
    rows = []
    metrics = []
    for strategy, strategy_frame in weights.groupby("strategy", sort=True):
        prev = pd.Series(0.0, index=TICKERS)
        for date, group in strategy_frame.groupby("date", sort=True):
            current = group.set_index("ticker")["weight"].reindex(TICKERS).fillna(0.0)
            realized = group.set_index("ticker")
["actual_future_return_21d"].reindex(TICKERS).fillna(0.0)
            gross = float((current * realized).sum())
            turnover = float(np.abs(current - prev).sum())
            net = gross - cost_rate * turnover
            rows.append(
                {
                    "date": pd.Timestamp(date),
                    "strategy": strategy,

```



```

        "gross_return": gross,
        "turnover": turnover,
        "transaction_cost_rate": cost_rate,
        "transaction_cost": cost_rate * turnover,
        "net_return": net,
    }
    )
    prev = current
    returns = pd.DataFrame(rows).sort_values(["strategy", "date"])
    for strategy, group in returns.groupby("strategy", sort=True):
        metrics.append(
            {
                "strategy": strategy,
                "return_type": "gross",
                "transaction_cost_rate": 0.0,
                **evaluate_returns(group["gross_return"], group["turnover"]),
            }
        )
    metrics.append(
        {
            "strategy": strategy,
            "return_type": "net",
            "transaction_cost_rate": cost_rate,
            **evaluate_returns(group["net_return"], group["turnover"]),
        }
    )
    return returns, pd.DataFrame(metrics).sort_values(["strategy", "return_type"])

```

B.4 AI 风险档位组合生成与校验

AI 组合层使用显式风险约束和程序化校验。低、中、高三档组合分别约束年化波动率和单 ETF 上限；若波动率约束不可行，系统回退至同一单资产上限下的最低波动组合并保留原因。

```

def optimize_profile(
    expected_annual: np.ndarray,
    cov: np.ndarray,
    profile: dict[str, Any],
) -> tuple[np.ndarray, bool, str]:
    n = len(TICKERS)
    max_weight = float(profile["single_etf_weight_max"])
    vol_cap = float(profile["annual_volatility_max"])
    x0 = min_volatility_weights(cov, max_weight)
    bounds = [(0.0, max_weight) for _ in range(n)]
    constraints = [
        {"type": "eq", "fun": lambda w: np.sum(w) - 1.0},
        {"type": "ineq", "fun": lambda w: vol_cap - portfolio_volatility(w, cov)},
    ]

    def objective(w: np.ndarray) -> float:
        diversification_penalty = 0.005 * float(np.sum(w**2))
        return -float(expected_annual @ w) + diversification_penalty

    result = minimize(
        objective,
        x0,
        method="SLSQP",
        bounds=bounds,
        constraints=constraints,
        options={"maxiter": 1000, "ftol": 1e-12},
    )
    if result.success:
        weight = np.clip(result.x, 0.0, max_weight)
        weight = weight / weight.sum()
        if portfolio_volatility(weight, cov) <= vol_cap + 1e-5:
            return weight, True, "risk_constraint_satisfied"

```

```

        fallback = x0
        feasible = portfolio_volatility(fallback, cov) <= vol_cap + 1e-5
        return fallback, feasible, "fallback_to_minimum_volatility"

def build_ai_weights(
    returns: pd.DataFrame,
    scores: pd.DataFrame,
) -> tuple[pd.DataFrame, list[dict[str, Any]]]:
    rows = []
    decisions = []
    for date in sorted(scores["date"].unique()):
        date = pd.Timestamp(date)
        cov = covariance_for_date(returns, date)
        expected = expected_annual_returns_for_date(returns, scores, date)
        actual = (
            scores[scores["date"] == date]
            .set_index("ticker")
            .reindex(TICKERS)["actual_future_return_21d"]
            .fillna(0.0)
            .to_numpy(dtype=float)
        )
        for profile_name, profile in RISK_PROFILES.items():
            weight, feasible, fallback_reason = optimize_profile(expected, cov,
profile)
            for ticker, ticker_weight, ticker_expected, ticker_actual in zip(
                TICKERS,
                weight,
                expected,
                actual,
                strict=False,
            ):
                rows.append(
                    {
                        "date": date,
                        "strategy": f"ai_{profile_name}",
                        "risk_profile": profile_name,
                        "ticker": ticker,
                        "weight": float(ticker_weight),
                        "expected_annual_return_asset": float(ticker_expected),
                        "actual_future_return_21d": float(ticker_actual),
                        "risk_constraint_met": bool(feasible),
                        "fallback_reason": fallback_reason,
                    }
                )
                decisions.append(
                    {
                        "date": date.date().isoformat(),
                        "risk_profile": profile_name,
                        "risk_constraint_met": bool(feasible),
                        "fallback_reason": fallback_reason,
                        "expected_annual_return": float(expected @ weight),
                        "expected_annual_volatility": portfolio_volatility(weight, cov),
                        "weights": {ticker: float(value) for ticker, value in zip(TICKERS,
weight, strict=False)},
                    }
                )
    return pd.DataFrame(rows), decisions

def validate_ai_outputs(
    ai_weights: pd.DataFrame,
    ai_json: dict[str, Any],
    metrics: pd.DataFrame,
) -> dict[str, Any]:
    weight_checks = []
    for (date, strategy), group in ai_weights.groupby(["date", "strategy"]):
        risk_profile = strategy.replace("ai_", "")
        max_weight = RISK_PROFILES[risk_profile]["single_etf_weight_max"]
        weight_checks.append(
            {
                "date": pd.Timestamp(date).date().isoformat(),
                "strategy": strategy,
                "sum_ok": bool(abs(group["weight"].sum() - 1.0) <= 1e-6),
                "long_only_ok": bool(group["weight"].min() >= -1e-8),
            }
        )

```

```

        "single_weight_cap_ok": bool(group["weight"].max() <= max_weight +
1e-8),
        "volatility_cap_ok": bool(group["risk_constraint_met"].all()),
        "fallback_reason": str(group["fallback_reason"].iloc[0]),
    }
)
numeric_cols = [
    "total_return",
    "annual_return",
    "annual_volatility",
    "sharpe",
    "sortino",
    "max_drawdown",
    "win_rate",
    "average_turnover",
]
return {
    "ai_weight_checks_ok": bool(
        all(
            x["sum_ok"] and x["long_only_ok"] and x["single_weight_cap_ok"]
            for x in weight_checks
        )
    ),
    "ai_risk_constraints_all_met": bool(all(x["volatility_cap_ok"] for x in
weight_checks)),
    "ai_risk_constraints_or_fallbacks_ok": bool(
        all(x["volatility_cap_ok"] or x["fallback_reason"] ==
"fallback_to_minimum_volatility" for x in weight_checks)
    ),
    "risk_constraint_fallback_count": int(sum(not x["volatility_cap_ok"] for x in
weight_checks)),
    "ai_json_profiles": [profile["profile"] for profile in
ai_json["risk_profiles"]],
    "metrics_finite_ok":
bool(np.isfinite(metrics[numeric_cols].to_numpy(dtype=float)).all()),
    "weight_checks": weight_checks,
}

```

B.5 独立回测、未来函数审计与图表输出

独立回测脚本不复用主脚本的指标结果，而是读取已有权重、收益和特征元数据重

新计算净值、回撤、评分排序、成本敏感性、未来函数审计和全部图表。

```

def build_lookahead_audit(
    root: Path,
    input_dir: Path,
    backtest_returns: pd.DataFrame,
    feature_metadata: dict[str, Any],
    validation_summary: dict[str, Any],
) -> dict[str, Any]:
    modeling_path = root / "data" / "processed" /
"modeling_dataset_monthly_rebalance.csv"
    require_file(modeling_path)
    monthly = pd.read_csv(modeling_path, parse_dates=["date"])
    scores_path = input_dir / "spo_prediction_scores.csv"
    require_file(scores_path)
    scores = pd.read_csv(scores_path, parse_dates=["date"])
    test_rows = monthly[monthly["split"] == "test"]
    train_valid_rows = monthly[monthly["split"].isin(["train", "valid"])]

    source = (root / "src" / "reproduce_spo_paper.py").read_text(encoding="utf-8")
    feature_config = feature_metadata.get("feature_config", {})
    quality_summary = feature_metadata.get("quality_summary", {})
    spo_data = validation_summary.get("spo_metadata", {}).get("data", {})

    backtest_dates =
pd.Series(backtest_returns["date"].drop_duplicates()).sort_values()
    test_dates = pd.Series(scores["date"].drop_duplicates()).sort_values()
    same_test_start = (

```

```

        bool(backtest_dates.iloc[0] == test_dates.iloc[0])
        if len(backtest_dates) and len(test_dates)
        else False
    )
    same_test_end = (
        bool(backtest_dates.iloc[-1] == test_dates.iloc[-1])
        if len(backtest_dates) and len(test_dates)
        else False
    )

    checks = [
        {
            "rule": "t month-end features only use data observed at or before sample
date",
            "status": feature_config.get("feature_lag_rule",
"" ).lower().startswith("all features use"),
            "evidence": feature_config.get("feature_lag_rule", ""),
        },
        {
            "rule": "t month-end features predict t+1 month / forward 21-trading-day
returns",
            "status": "Forward 21" in feature_config.get("label_rule", ""),
            "evidence": feature_config.get("label_rule", ""),
        },
        {
            "rule": "portfolio return uses the next-period realized return already
stored as future_return_21d",
            "status": same_test_start and same_test_end,
            "evidence": (
                f"backtest_dates={backtest_dates.iloc[0].date()}..
{backtest_dates.iloc[-1].date()}", "
                f"test_label_dates={test_dates.iloc[0].date()}..
{test_dates.iloc[-1].date()}"
                if len(backtest_dates) and len(test_dates)
                else "missing dates"
            ),
        },
        {
            "rule": "StandardScaler is fit on train split only, then transforms
validation and test",
            "status": (
                "train_panel = monthly_panel[monthly_panel[\"split\"] == \"train\"]"
in source
                and "fit_scaler=True" in source
                and "fit_scaler=False" in source
                and validation_summary.get("spo_metadata", {})
.get("data", {})
.get("scaler_fit_rule", "")
.startswith("StandardScaler is fit on train split only")
            ),
            "evidence": (
                validation_summary.get("spo_metadata", {})
.get("data", {})
.get("scaler_fit_rule", "missing scaler metadata")
            ),
        },
        {
            "rule": "split dates keep test data after training/validation period",
            "status": (
                not train_valid_rows.empty
                and not test_rows.empty
                and train_valid_rows["date"].max() < test_rows["date"].min()
            ),
            "evidence": (
                f"train_valid_end={train_valid_rows['date'].max().date()}", "
                f"test_start={test_rows['date'].min().date()}"
            ),
        },
    ]
    all_passed = bool(all(item["status"] for item in checks))
    return {
        "generated_at_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "all_checks_passed": all_passed,
    }

```

```

        "input_dir": str(input_dir),
        "feature_count": quality_summary.get("feature_count",
len(quality_summary.get("feature_names", []))),
        "split_rule": feature_config.get("split_rule", ""),
        "model_fit_rule": spo_data.get("model_fit_rule"),
        "metadata_test_start": spo_data.get("test_start"),
        "metadata_test_end": spo_data.get("test_end"),
        "checks": checks,
    }

def evaluate_returns(returns: pd.Series, turnover: pd.Series) -> dict[str, Any]:
    clean_returns = returns.fillna(0.0).astype(float)
    n = len(clean_returns)
    wealth = float((1.0 + clean_returns).prod()) if n else 1.0
    annual_return = wealth ** (MONTHS_PER_YEAR / n) - 1.0 if n else 0.0
    annual_vol = (
        float(clean_returns.std(ddof=1) * np.sqrt(MONTHS_PER_YEAR)) if n > 1 else 0.0
    )
    downside = clean_returns[clean_returns < 0]
    downside_vol = (
        float(downside.std(ddof=1) * np.sqrt(MONTHS_PER_YEAR)) if len(downside) > 1
    else 0.0
    )
    best_month = float(clean_returns.max()) if n else 0.0
    worst_month = float(clean_returns.min()) if n else 0.0
    mdd = max_drawdown(clean_returns)
    return {
        "months": int(n),
        "total_return": wealth - 1.0,
        "annual_return": annual_return,
        "annual_volatility": annual_vol,
        "sharpe": annual_return / annual_vol if annual_vol > 0 else 0.0,
        "sortino": annual_return / downside_vol if downside_vol > 0 else 0.0,
        "max_drawdown": mdd,
        "calmar": annual_return / abs(mdd) if mdd < 0 else 0.0,
        "longest_drawdown_months": longest_drawdown_months(clean_returns),
        "win_rate": float((clean_returns > 0).mean()) if n else 0.0,
        "best_month": best_month,
        "worst_month": worst_month,
        "average_monthly_return": float(clean_returns.mean()) if n else 0.0,
        "average_turnover": float(turnover.mean()) if len(turnover) else 0.0,
        "final_wealth": wealth,
    }

def calculate_metrics(backtest_returns: pd.DataFrame, cost_rate: float) ->
pd.DataFrame:
    rows = []
    for strategy, group in backtest_returns.groupby("strategy", sort=True):
        for return_type, column, tx_rate in [
            ("gross", "gross_return", 0.0),
            ("net", "net_return", cost_rate),
        ]:
            rows.append(
                {
                    "strategy": strategy,
                    "strategy_group": MODEL_GROUPS.get(strategy,
BENCHMARKS.get(strategy, "other")),
                    "return_type": return_type,
                    "transaction_cost_rate": tx_rate,
                    "strategy_note": STRATEGY_NOTES.get(strategy, ""),
                    **evaluate_returns(group[column], group["turnover"]),
                }
            )
    return pd.DataFrame(rows).sort_values(["return_type",
"strategy"]).reset_index(drop=True)

def rank_models(metrics: pd.DataFrame) -> pd.DataFrame:
    net = metrics[
        (metrics["return_type"] == "net") & (metrics["strategy_group"] != "benchmark")
    ].copy()
    net["quality_score"] = (
        0.25 * normalize_score(net["annual_return"])

```

```

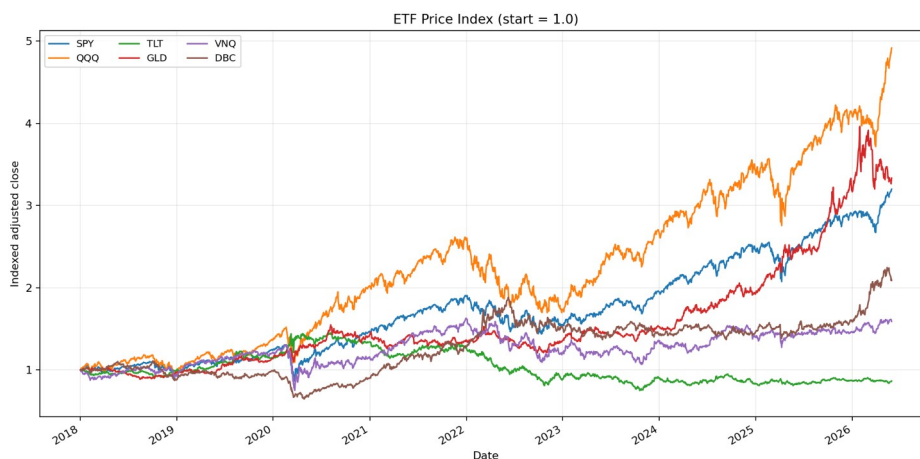
        + 0.20 * normalize_score(net["sharpe"])
        + 0.20 * normalize_score(net["max_drawdown"], higher_is_better=True)
        + 0.15 * normalize_score(net["win_rate"])
        + 0.10 * normalize_score(net["average_turnover"], higher_is_better=False)
        + 0.10 * normalize_score(net["calmar"])
    )
    net["rank"] = net["quality_score"].rank(ascending=False,
method="first").astype(int)
    columns = [
        "rank",
        "strategy",
        "strategy_group",
        "quality_score",
        "total_return",
        "annual_return",
        "annual_volatility",
        "sharpe",
        "calmar",
        "max_drawdown",
        "win_rate",
        "average_turnover",
    ]
    return net.sort_values("rank")[columns].reset_index(drop=True)

def save_all_plots(
    price_index: pd.DataFrame,
    curves: pd.DataFrame,
    weights: pd.DataFrame,
    metrics: pd.DataFrame,
    importance: pd.DataFrame,
    output_dir: Path,
) -> dict[str, str]:
    return {
        "etf_price_index": save_etf_price_chart(price_index, output_dir),
        "strategy_cumulative_returns": save_strategy_equity_chart(curves, output_dir),
        "strategy_drawdowns": save_strategy_drawdown_chart(curves, output_dir),
        "ai_monthly_weight_heatmap": save_ai_weight_heatmap(weights, output_dir),
        "random_forest_feature_importance": save_rf_importance_chart(importance,
output_dir),
        "strategy_performance_table": save_performance_table_chart(metrics,
output_dir),
    }

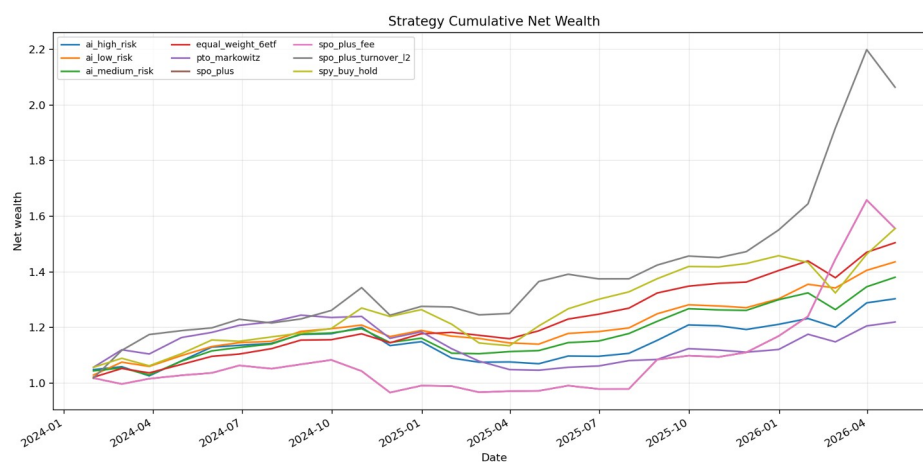
```

附录 C 附图汇编

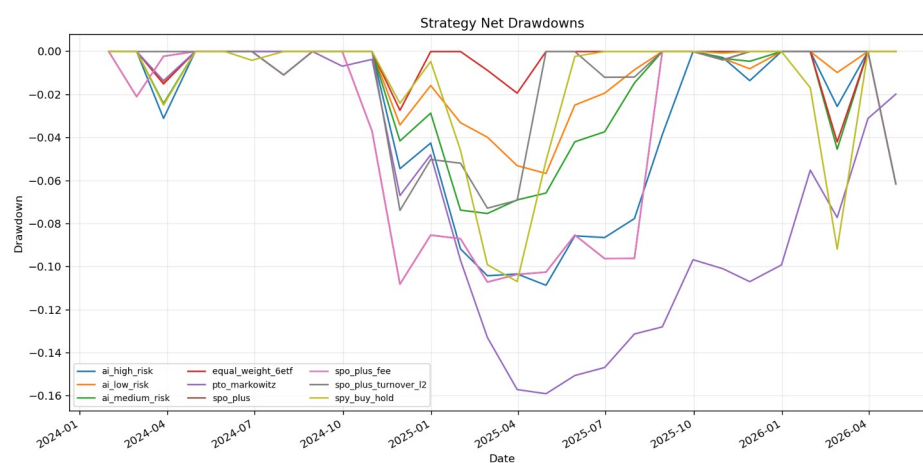
附录 C 汇集独立回测脚本生成的全部可视化图像。正文已引用主要图表，附录再次集中列示，便于审阅者核对图像来源、编号和含义。



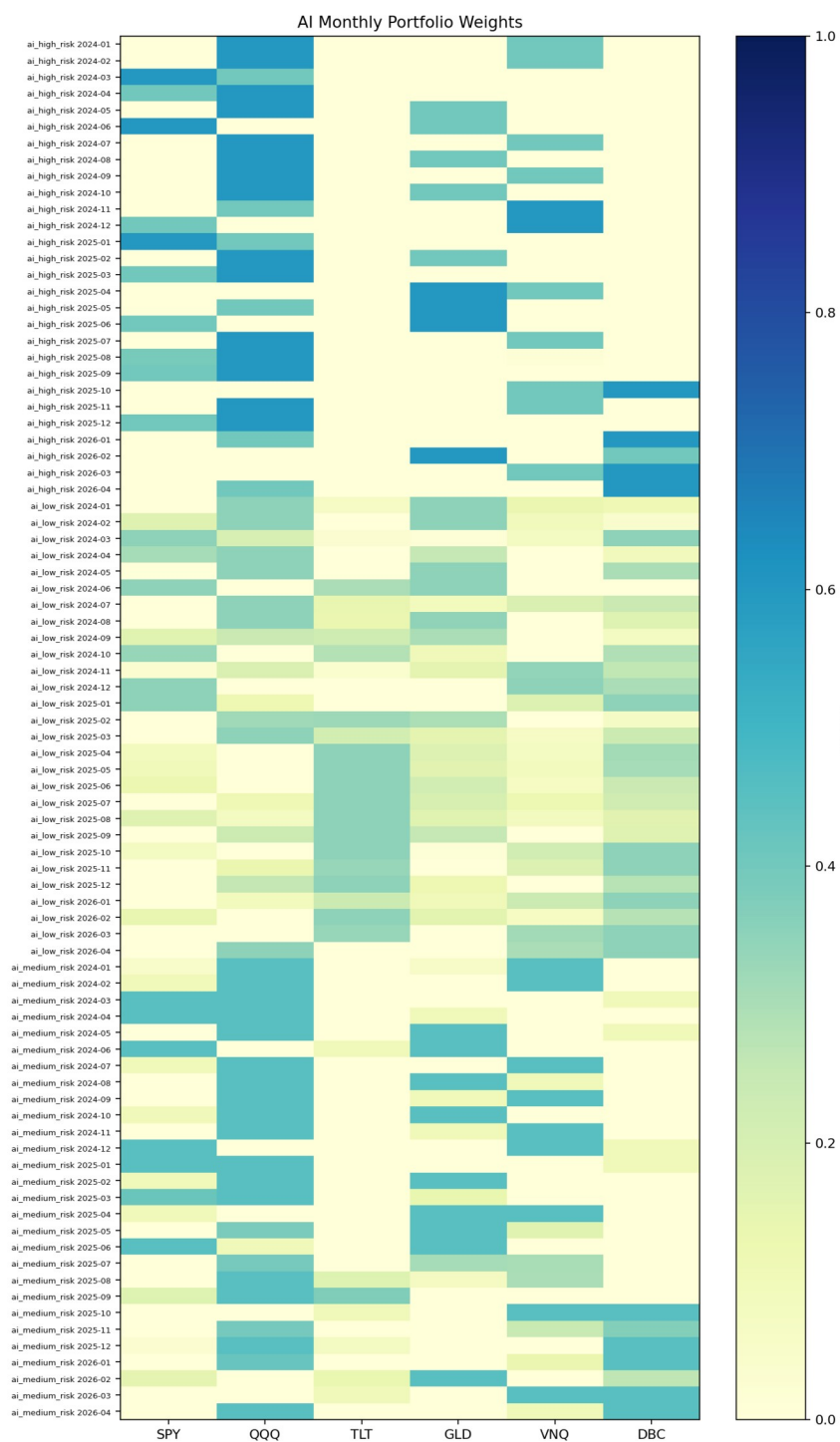
附图 C-1 ETF 价格指数走势 Appendix Figure C-1 ETF price index



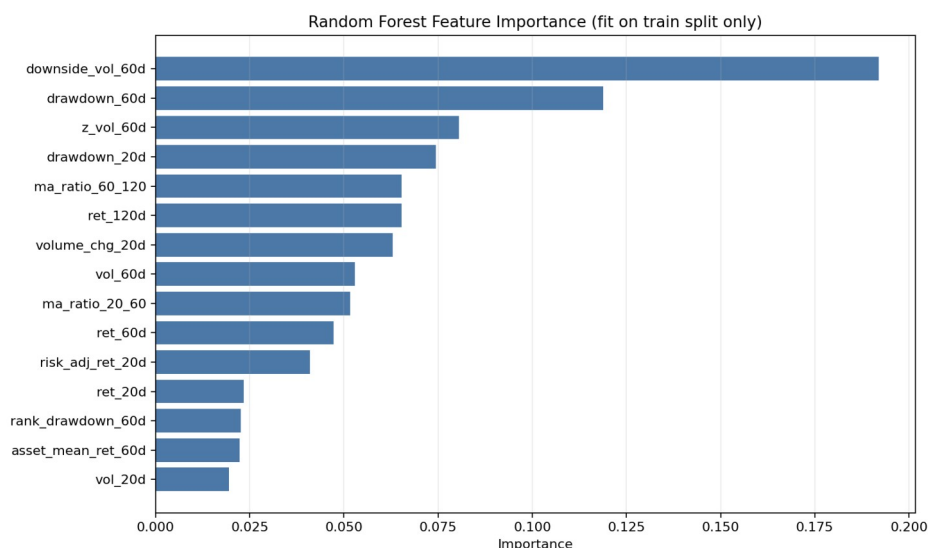
附图 C-2 策略累计收益曲线 Appendix Figure C-2 Strategy cumulative returns



附图 C-3 策略回撤曲线 Appendix Figure C-3 Strategy drawdowns



附图 C-4 AI 组合月度权重热力图 Appendix Figure C-4 AI monthly allocation heatmap



附图 C-5 随机森林特征重要性 Appendix Figure C-5 Random forest feature importance

Strategy Performance Comparison (Net)

strategy	annual_return	annual_volatility	sharpe	max_drawdown	win_rate	average_turnover	calmar
spo_plus_turnover_12	36.41%	18.16%	2.00	-7.38%	75.00%	75.00%	4.93
spo_plus	20.87%	18.14%	1.15	-10.81%	67.86%	132.14%	1.93
spo_plus_fee	20.87%	18.14%	1.15	-10.81%	67.86%	132.14%	1.93
spy_buy_hold	20.86%	13.74%	1.52	-10.69%	67.86%	3.57%	1.95
equal_weight_getf	19.15%	7.62%	2.51	-4.20%	82.14%	3.57%	4.56
ai_low_risk	16.78%	7.49%	2.24	-5.66%	67.86%	69.01%	2.96
ai_medium_risk	14.83%	9.40%	1.58	-7.52%	75.00%	91.16%	1.97
ai_high_risk	12.03%	10.41%	1.16	-10.86%	67.86%	103.57%	1.11
pto_markowitz	8.90%	10.84%	0.82	-15.90%	64.29%	126.39%	0.56

附图 C-6 策略绩效对比表 Appendix Figure C-6 Strategy performance table

附录 D 附表索引与主要输出表

附录 D 列示本项目生成的主要数据表、JSON、图像和报告文件，并补充若干关键输出预览。完整数据以对应文件为准。

附表 D-1 输出产物索引 Appendix Table D-1 Output artifact index

路径	类型	规模或结构	大小	用途
outputs/tables/ai_prompt_payload.json	JSON	JSON 对象；一级键： generated_for_decision_date, system_prompt, user_prompt_template	17.9KB	AI 风险组合提示词输入、

		, structured_inputs, required_output_schema		约束与 候选组 合载荷 三档 AI 风 险组合 逐月收 益与换 手
outputs/tables/ai_risk_profile_backtest_returns.csv	CSV	84 行 x 7 列	9.2KB	最新三 档风险 组合 JSON 交付结 果
outputs/tables/ai_risk_profile_portfolios.json	JSON	JSON 对象；一级 键：generated_at_utc, decision_date, ai_generation_mode, source_files, prompt_inputs_summary	7.4KB	AI 组 合权 重、风 险约束 与指标 有限性 校验 三档 AI 风 险组合 逐月
outputs/tables/ai_risk_profile_validation.json	JSON	JSON 对象；一级 键：generated_at_utc, artifacts, validation, disclaimer	22.9KB	ETF 权 重明细 ETF 原 始价格 与收益 率质量 检查明 细
outputs/tables/ai_risk_profile_weights.csv	CSV	504 行 x 9 列	62.0KB	下载配
outputs/tables/data_quality_check.csv	CSV	6 行 x 14 列	1.0KB	
outputs/tables/download_metadata.json	JSON	JSON 对象；一级	1.8KB	

		键： download_config, quality_summary, artifacts		置、数 据源、 备用通 道与质 量摘要 ETF 单 体风险 收益、 动量与 预测排 名 特征、 标签、 样本切 分与缺 失检查 摘要 主脚本 输出的 策略回 测指标 SPO/ PtO/基 准策略 逐月收 益 候选组 合预期 收益、 波动、 权重 JSON 与换手 Ridge 与 SPO+
outputs/tables/etf_risk_return_metrics.csv	CSV	6 行 x 13 列	1.5KB	
outputs/tables/feature_dataset_quality.json	JSON	JSON 对象；一级 键：feature_config, quality_summary, artifacts	3.0KB	
outputs/tables/portfolio_backtest_metrics.csv	CSV	18 行 x 13 列	3.6KB	
outputs/tables/spo_backtest_returns.csv	CSV	168 行 x 7 列	15.3KB	
outputs/tables/spo_candidate_portfolios.csv	CSV	168 行 x 10 列	37.3KB	
outputs/tables/spo_model_metrics.csv	CSV	4 行 x 6 列	0.5KB	

				预测层 诊断指 标 SPO、 PtO 与 基准策 略逐月 ETF 权 重 测试期 各 ETF 预测分 数与未 来收益 SPO 复 现实验 配置、 数据切 分、校 验与引 用信息 SPO+
outputs/tables/spo_portfolio_weights.csv	CSV	1008 行 x 7 列	117.8 KB	训练损 失、验 证 MSE 与 Rank IC 记 录 独立回 测总摘 要、最 佳策略 与主要 风险提
outputs/tables/spo_prediction_scores.csv	CSV	168 行 x 9 列	14.5KB	
outputs/tables/spo_reproduction_metadata.json	JSON	JSON 对象；一级 键：generated_at_utc, conda_environment, command, paper, implementation	25.3KB	
outputs/tables/spo_training_history.csv	CSV	13 行 x 4 列	0.8KB	
test/backtest_outputs/backtest_summary.json	JSON	JSON 对象；一级 键：generated_at_utc, input_dir, output_dir, benchmark, cost_rate	58.4KB	

				示 合并 SPO、 PtO、 基准与 AI 组 合后的 逐月收 益 独立回 测自动 生成的 审计报 告 未来函 数与时 间顺序 审计结 果 策略优 势、弱 点与推 荐用途 评估 策略质 量综合 评分排 序 随机森 林诊断 性特征 重要性 独立回 测总收 益、波
test/backtest_outputs/ combined_model_backtest_returns.csv	CSV	252 行 x 9 列	30.0KB	
test/backtest_outputs/ existing_model_backtest_report.md	MD	Markdown ; 130 行	9.8KB	
test/backtest_outputs/lookahead_audit.json	JSON	JSON 对象 ; 一级 键 : generated_at_utc, all_checks_passed, input_dir, feature_count, split_rule	1.5KB	
test/backtest_outputs/model_assessment.csv	CSV	7 行 x 6 列	1.5KB	
test/backtest_outputs/model_quality_ranking.csv	CSV	7 行 x 12 列	1.5KB	
test/backtest_outputs/ random_forest_feature_importance.csv	CSV	22 行 x 3 列	0.8KB	
test/backtest_outputs/strategy_backtest_metrics.csv	CSV	18 行 x 20 列	6.4KB	

				动、回 撤、换 手等指 标
test/backtest_outputs/ strategy_drawdown_curves.csv	CS V	252 行 x 4 列	11.9K B	各策略 逐月回 撤曲线
test/backtest_outputs/strategy_equity_curves.csv	CS V	252 行 x 10 列	37.2K B	各策略 逐月净 值曲线 相对等 权基准
test/backtest_outputs/strategy_excess_returns.csv	CS V	8 行 x 8 列	1.2KB	的超额 收益与 信息比 率
test/backtest_outputs/ transaction_cost_sensitivity.csv	CS V	54 行 x 17 列	14.8K B	不同交 易成本 假设下 的绩效 敏感性
test/backtest_outputs/plots/ ai_monthly_weight_heatmap.png	PN G	PNG 图像	202.9 KB	AI 三 档风险 组合逐 月权重 热力图
test/backtest_outputs/plots/etf_price_index.png	PN G	PNG 图像	237.4 KB	六只 ETF 归 一化价 格指数 走势
test/backtest_outputs/plots/ random_forest_feature_importance.png	PN G	PNG 图像	64.6K B	随机森 林诊断 性特征 重要性

test/backtest_outputs/plots/ strategy_cumulative_returns.png	PN G	PNG 图像	168.3 KB	图 主要策 略累计 净值曲 线
test/backtest_outputs/plots/ strategy_drawdowns.png	PN G	PNG 图像	227.4 KB	主要策 略回撤 曲线
test/backtest_outputs/plots/ strategy_performance_table.png	PN G	PNG 图像	114.2 KB	主要策 略绩效 表图像

附表 D-2 ETF 数据质量明细 Appendix Table D-2 ETF data quality details

ETF	起始日期	结束日期	价格行数	复权缺失	非正价格	区间总收 益	区间最大 回撤
SPY	2018-01-02	2026-05-29	2113	0	0	219.78%	-33.72%
QQQ	2018-01-02	2026-05-29	2113	0	0	391.48%	-35.12%
TLT	2018-01-02	2026-05-29	2113	0	0	-13.46%	-48.35%
GLD	2018-01-02	2026-05-29	2113	0	0	233.30%	-22.00%
VNQ	2018-01-02	2026-05-29	2113	0	0	59.05%	-42.40%
DBC	2018-01-02	2026-05-29	2113	0	0	108.92%	-41.71%

附表 D-3 模型质量综合排序 Appendix Table D-3 Model quality ranking

排 名	策略	策略组	质量 分	总收益	年化收 益	Sharpe	最大回 撤	平均换 手
1	SPO+换 手/L2 约 束	spo_model	0.924	106.36%	36.41%	2.00	-7.38%	75.00%
2	AI 低风险 组合	ai_risk_profile	0.677	43.62%	16.78%	2.24	-5.66%	69.01%
3	AI 中风险 组合	ai_risk_profile	0.571	38.08%	14.83%	1.58	-7.52%	91.16%
4	SPO+	spo_model	0.336	55.63%	20.87%	1.15	-10.81 %	132.14%

5	SPO+交易费	spo_model	0.336	55.63%	20.87%	1.15	-10.81%	132.14%
6	AI 高风险组合	ai_risk_profile	0.282	30.36%	12.03%	1.16	-10.86%	103.57%
7	PtO-Markowitz	predict_then_optimize	0.009	22.00%	8.90%	0.82	-15.90%	126.39%

附表 D-4 交易成本敏感性摘要 Appendix Table D-4 Transaction cost sensitivity summary

策略	成本率	总收益	年化收益	Sharpe	最大回撤	平均换手
6ETF 等权	0.00%	51.23%	19.40%	2.54	-4.20%	3.57%
6ETF 等权	0.10%	51.08%	19.35%	2.53	-4.20%	3.57%
6ETF 等权	0.50%	50.49%	19.15%	2.51	-4.20%	3.57%
6ETF 等权	1.00%	49.76%	18.90%	2.48	-4.20%	3.57%
PtO-Markowitz	0.00%	45.37%	17.39%	1.60	-12.81%	126.39%
PtO-Markowitz	0.10%	40.38%	15.64%	1.44	-13.34%	126.39%
PtO-Markowitz	0.50%	22.00%	8.90%	0.82	-15.90%	126.39%
PtO-Markowitz	1.00%	2.25%	0.96%	0.09	-19.99%	126.39%
SPO+	0.00%	86.88%	30.73%	1.72	-8.91%	132.14%
SPO+	0.10%	80.19%	28.71%	1.61	-9.29%	132.14%
SPO+	0.50%	55.63%	20.87%	1.15	-10.81%	132.14%
SPO+	1.00%	29.37%	11.67%	0.63	-16.80%	132.14%
SPO+换手/L2 约束	0.00%	129.06%	42.65%	2.42	-6.38%	75.00%
SPO+换手/L2 约束	0.10%	124.35%	41.38%	2.33	-6.58%	75.00%
SPO+换手/L2 约束	0.50%	106.36%	36.41%	2.00	-7.38%	75.00%
SPO+换手/L2 约束	1.00%	85.72%	30.39%	1.61	-10.64%	75.00%

附表 D-5 策略评估与推荐用途 Appendix Table D-5 Strategy assessment and recommended use

排名	策略	策略组	优势	弱点	推荐用途
1	SPO+换手/L2 约束	spo_model	年化收益领先；风险调整收益强；正则化显著	主要风险来自样本期较短	适合作为 SPO 类策略的生产化起点，但需要继续降低换

			改善基础		
			SPO+ 的净		手。
			值表现		
			年化收益中		
			等偏稳；风		
			险调整收益		
2	AI 低风险组 合	ai_risk_profile	强；换手相 对可控；风 险约束让波 动和回撤更 平滑	主要风险来 自样本期较 短	适合作为低波 动核心组合或 防守档位。
			年化收益中		适合作为当前
			等偏稳；风		样本下的主组
3	AI 中风险组 合	ai_risk_profile	险约束让波 动和回撤更 平滑	主要风险来 自样本期较 短	合候选，收益 和风险比较均 衡。
				回撤压力	更适合保留为
4	SPO+	spo_model	年化收益领 先	大；换手高 且交易成本 敏感	消融实验基 线，而非直接 实盘化。
				回撤压力	
				大；换手高 且交易成本 敏感；当前 输出与基础 SPO+ 几乎 重合，费用 项没有形成 有效约束	更适合保留为 消融实验基 线，而非直接 实盘化。
5	SPO+交易费	spo_model	年化收益领 先		
			年化收益中		
			等偏稳；风		
6	AI 高风险组 合	ai_risk_profile	险约束让波 动和回撤更 平滑	回撤压力 大；换手高 且交易成本 敏感	适合愿意承受 更高波动、追 求更高收益弹 性的配置。
7	PtO-	predict_then_optimize	没有明显优	净收益不	适合作为传统

Markowitz	势	足；Sharpe 偏低；回撤 压力大；换 手高且交易 成本敏感	PtO/Markowitz 对照模型，不 宜忽略成本。
-----------	---	---	-----------------------------------

附表 D-6 AI 风险组合校验摘要 Appendix Table D-6 AI risk profile validation summary

检查项	结果
权重和、非负、单资产上限校验	True
全部风险约束直接满足	False
风险约束或回退机制有效	True
风险约束回退次数	2
组合指标有限性校验	True
纳入校验的风险档位	low_risk, medium_risk, high_risk